

# Tensor networks as surrogate models of training dynamics

Research note, October 7, 2026

## Abstract

This note motivates tree tensor networks (TTNs) as surrogate models of the training dynamics of deep neural networks, and makes the underlying mathematics precise. We define tensor networks, TTNs, their edge matricizations, hierarchical ranks and hierarchical singular value decomposition (HSVD), and compare the relevant notions of tensor rank. A deep network with polynomial activations is exactly a weight-tied TTN evaluated on copies of the homogenized input. This suggests reading the HSVD of that TTN as a gauge-invariant description of the hierarchical features learned by the network, and its hierarchical ranks as a measure of learning capacity; we make both notions precise and discuss their limits. Taylor-expanding the activations of a smooth network at a training checkpoint gives a local TTN surrogate, whose fidelity to the original training dynamics is controlled by prediction and Jacobian errors. We state a global and a local surrogate hypothesis, the open questions they raise, and the experiments that would test them.

## Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>2</b>  |
| <b>2</b> | <b>Theoretical background</b>                                    | <b>4</b>  |
| 2.1      | Tensors, multilinear maps and matricizations . . . . .           | 5         |
| 2.2      | Tensor networks and tree tensor networks . . . . .               | 5         |
| 2.3      | Edge cuts, bond dimensions and hierarchical ranks . . . . .      | 7         |
| 2.4      | The hierarchical singular value decomposition . . . . .          | 8         |
| 2.5      | Notions of rank . . . . .                                        | 10        |
| 2.6      | Deep polynomial networks as tree tensor networks . . . . .       | 11        |
| 2.6.1    | Monomial activations: tied TTNs on copies of the input . . . . . | 11        |
| 2.6.2    | General polynomial activations: homogenization . . . . .         | 12        |
| 2.6.3    | Hierarchical features and learning capacity . . . . .            | 15        |
| 2.7      | Surrogate model of a deep neural network . . . . .               | 18        |
| <b>3</b> | <b>Training dynamics of TTNs</b>                                 | <b>22</b> |
| <b>4</b> | <b>How to compare TTNs and DNNs</b>                              | <b>22</b> |
| <b>5</b> | <b>Experiments (slop)</b>                                        | <b>24</b> |

# 1 Introduction

In this research note I motivate tree tensor networks (TTNs) as a surrogate model of the training dynamics of deep neural networks (DNNs). I also suggest experiments to check whether TTNs are a promising surrogate for the training dynamics of more realistic neural networks. Section 2 develops the mathematical background, and the experiments are collected in Section 5.

**Why training dynamics?** To be brief, my current position is that AI safety is bottlenecked by understanding. Interpretability as a field remains shallow, in the sense that current methods explain little about the behaviour of current models, or are unlikely to scale. One of the main reasons for this state of affairs is that we are still confused about fundamental questions on how neural networks work. One such question is *what kind of structure neural networks learn*. Extracting this structure from a network at the end of training seems extremely challenging. The bet behind studying training dynamics is that internal structure decomposes more naturally during training, as it is progressively learned.

**Why tensor networks?** We have a good understanding of deep linear networks (DLNs). For whitened inputs and a suitably aligned initialization, Saxe et al. (2014) solved the gradient-flow dynamics of DLNs exactly: the singular modes of the input–output map decouple and are learned one after the other, each after a plateau, stronger modes first (see also Saxe et al., 2019). From a small initialization, DLNs and diagonal linear networks follow saddle-to-saddle dynamics in which the rank (or the support) of the learned map increases step by step (Jacot et al., 2021; Pesme and Flammarion, 2023), and similar dynamics explain a simplicity bias across architectures (Zhang et al., 2026). We are, however, still far from understanding more complex architectures such as transformers. Even the learning dynamics of deep networks with quadratic activations remain largely open, although there are interesting results for shallow ones (e.g. Du and Lee, 2018; Sarao Mannelli et al., 2020).

TTNs provide an interesting model of more complex deep networks (Furman et al., 2026). A TTN is a tree of multilinear maps (Definition 2.7); it generalizes DLNs, which are chains, and Tucker decompositions, which are stars. Unlike a DLN, a TTN computes nonlinear functions of its inputs: it can implement any read-once Boolean formula, and it can be evaluated in time polynomial in the size of the tree and in the bond dimensions. Consequently, TTNs contain targets that are easy to evaluate but cannot be learned by gradient descent in polynomial time, such as parities, for the same reason as neural networks. Nevertheless their loss landscape is conditionally benign, and the difficulty can instead come from high-order saddle points, which are located at rank-deficient parameters (Furman et al., 2026, Theorems 4.3 and 5.2, Proposition 5.3). Hierarchical tensor formats can moreover be exponentially more compact than shallow (CP) ones (Cohen et al., 2016). At the same time, TTNs look more interpretable than generic networks because they come with a canonical, gauge-invariant hierarchical singular value decomposition (HSVD, Section 2.4). Empirically, TTNs trained from a small initialization undergo saddle-to-saddle dynamics, i.e. they learn features sequentially, consistently with the incremental hierarchical-rank bias of hierarchical tensor factorizations (Razin et al., 2022) and with the rank-deficient saddles of Furman et al. (2026). TTNs are also more expressive than DLNs, and they can display *soft hierarchical learning*: learning some features accelerates the learning of subsequent features. The following definition makes this precise.

**Definition 1.1** (Hard and soft acceleration; after Diagonal TTN note, 2026, Definition 3.1). Let  $q_B(\vartheta)$  be an observable measuring the acquisition of a feature  $B$ , with target value  $q_B^* > 0$ , and fix

a threshold  $\delta \in (0, 1)$ . For an intervention  $\mathcal{I}$  on training (for example, clamping the parameters that compute another feature  $A$  to a learned or to a blocked value, all else being equal), let

$$\tau_B(\delta \mid \mathcal{I}) = \inf\{t \geq 0 : q_B(\vartheta_{\mathcal{I}}(t)) \geq \delta q_B^*\} \in [0, \infty].$$

Learning  $A$  *accelerates* learning  $B$  if  $\tau_B(\delta \mid A \text{ learned}) < \tau_B(\delta \mid A \text{ blocked})$ . The acceleration is *hard* if  $\tau_B(\delta \mid A \text{ blocked}) = \infty$ , and *soft* if both times are finite.

In a DLN with decoupled modes, the only such effect is the symmetric facilitation between the factors of a single mode. In TTNs, acceleration between distinct features can be computed exactly in simple cases (Diagonal TTN note, 2026, Sections 5–6): readouts of nested sub-products (“staircases”) produce hard prerequisite gating, and residual paths produce soft acceleration. For instance, in a three-leaf channelwise residual TTN with residual strength  $\rho$  and composition strength  $\gamma$ , learning the lower core (teacher value  $\alpha$ ) divides the acquisition time of the upper core by  $1 + \gamma^2 \alpha^2 / (2\rho^2)$  (Diagonal TTN note, 2026, Proposition 6.1). Expressivity alone is not the mechanism: with fixed orthogonal leaf features, a homogeneous diagonal TTN has exactly the amplitude dynamics of a diagonal DLN, and acceleration comes from parameters shared by several output features (Diagonal TTN note, 2026, Proposition 4.1 and Section 7). For a broader review of hierarchical feature learning and of the observables proposed to track it, see Literature review (2026).

**Two hypotheses.** While TTNs have properties that make them attractive as a model of deep networks, we would like to first establish how far their training dynamics is from that of more realistic architectures. If TTNs are a good model of the training dynamics of neural networks, then according to a range of observables the training dynamics of TTNs should track the training dynamics of more realistic architectures.

To state this precisely, fix a distribution  $\nu$  of input–label pairs  $(x, y) \in \mathbb{R}^{d_0} \times \mathbb{R}$ , with input marginal  $\nu_x$ , the square loss  $\mathcal{L}(f) = \frac{1}{2} \mathbb{E}_\nu[(f(x) - y)^2]$ , and an optimizer with its hyperparameters, for instance full-batch gradient descent  $\vartheta_{n+1} = \vartheta_n - \eta_n \nabla_{\vartheta} \mathcal{L}(f_{\vartheta_n})$ , or its gradient-flow limit in the time variable  $t = \sum_n \eta_n$ . Training a model class  $\vartheta \mapsto f_{\vartheta}$  from an initialization produces a trajectory  $t \mapsto f_{\vartheta(t)}$ . By an *observable* we mean a quantity that can be evaluated along the trajectory of any architecture. The most robust observables are computed from the predictors  $f_{\vartheta(t)}$  alone: the loss  $\mathcal{L}(f_{\vartheta(t)})$ ; the function-space distance  $\|f_{\vartheta(t)} - g_{\vartheta(t)}\|_{L^2(\nu_x)}$  between two models; and, in a teacher–student setting where the target  $f^* = \sum_{m=1}^M \sqrt{\lambda_m} \chi_m$  is built from features  $\chi_m$  that are orthonormal in  $L^2(\nu_x)$ , the feature coefficients, the acquisition times and the order of acquisition,

$$a_m(t) = \frac{\mathbb{E}_{\nu_x}[f_{\vartheta(t)}(x) \chi_m(x)]}{\sqrt{\lambda_m}}, \quad \tau_m(\delta) = \inf\{t \geq 0 : a_m(t) \geq \delta\} \quad (0 < \delta < 1), \quad (1.1)$$

the order being the permutation of  $\{1, \dots, M\}$  that sorts  $\tau_1(\delta), \dots, \tau_M(\delta)$ . Other observables depend on the parameterization but still live in a space common to all architectures, such as the empirical tangent kernel on a fixed set of inputs (Remark 1.4). Quantities attached to a particular representation, such as the HSVD of the TTN of Section 2.6, are diagnostics rather than observables: they can be compared across models only through a common representation (Remark 2.39). The hypothesis can be formulated at two levels. The first one is global.

**Hypothesis 1.2** (Global surrogate). Fix the data, the loss function and the hyperparameters, a family of observables, a tolerance  $\varepsilon > 0$  and a training horizon  $t_{\max}$ . There exist a TTN architecture and an initialization scheme such that, for typical initializations of both models, the observables of the TTN trajectory  $g_{\vartheta(t)}$  and of the DNN trajectory  $f_{\vartheta(t)}$  differ by at most  $\varepsilon$  for all  $t \in [0, t_{\max}]$ , in the same time variable and without post hoc time warping, and discrete observables, such as the order in which features are learned, coincide.

The second one is local. Even if the global version fails, we could still use TTNs as *local* surrogates of the training dynamics. Consider a DNN with smooth activation functions and Taylor expand its activations to some order around a training checkpoint. The resulting network is polynomial and, as shown in Section 2.6, it can be represented exactly as a TTN.

**Hypothesis 1.3** (Local surrogate). Let  $\theta_j = \theta(t_j)$  be a checkpoint of DNN training and  $\hat{f}_j$  the TTN surrogate built at  $\theta_j$  (Definition 2.44). Fix a tolerance  $\varepsilon > 0$  and a horizon  $H > 0$  that is not small compared with the time scale on which features are acquired. When the surrogate is started from  $\theta_j$ , with the same optimizer state, data and hyperparameters as the DNN, its trajectory  $\tilde{\theta}$  satisfies

$$\sup_{0 \leq s \leq H} \|f_{\theta(t_j+s)} - \hat{f}_j(\tilde{\theta}(t_j+s), \cdot)\|_{L^2(\nu_x)} \leq \varepsilon,$$

and the chosen observables agree within  $\varepsilon$  on  $[t_j, t_j + H]$ .

If  $\varepsilon$  exceeds the initial prediction error, some positive horizon always exists by continuity; the content of the hypothesis is that the horizon is long enough to cover non-trivial learning, such as the acquisition of a feature. A sequence of such local surrogates, rebuilt at successive checkpoints, could provide an interpretable “atlas” of training even when no single TTN is globally faithful (Surrogate note, 2026).

Note that even if these hypotheses fail, solving the training dynamics of TTNs could still be highly instructive in the same way that studying the training dynamics of DLNs is instructive although their training dynamics differs from more realistic architectures. For example the saddle to saddle dynamics regime is a regime that we can find in more realistic architecture and they teach us about the theoretical gap between simple toy models and more realistic architectures

**Remark 1.4** (Comparing different parameterizations). A DNN and a TTN have different parameters, but their dynamics can be compared in function space. Let  $x_1, \dots, x_n$  be the training inputs,  $F_\theta = (f_\theta(x_i))_{i \leq n}$ ,  $J_\theta = D_\theta F_\theta$ , and  $K_\theta = \frac{1}{n} J_\theta J_\theta^\top \in \mathbb{R}^{n \times n}$  the empirical tangent kernel (Jacot et al., 2018); define  $\tilde{F}_\vartheta$  and  $\tilde{K}_\vartheta$  in the same way for the TTN. Under full-batch gradient flow on the empirical square loss,  $\dot{F}_\theta = -K_\theta(F_\theta - y)$ , hence

$$\frac{d}{dt}(F_\theta - \tilde{F}_\vartheta) = -K_\theta(F_\theta - \tilde{F}_\vartheta) - (K_\theta - \tilde{K}_\vartheta)(\tilde{F}_\vartheta - y).$$

Function-space agreement is therefore plausible when the two models have close predictions and close residual-weighted tangent kernels (Surrogate note, 2026, Section 4.3). The two Jacobians need not have the same number of columns; their kernels are both  $n \times n$ .

**Outline.** Section 2 defines tensors, tensor networks and TTNs (Sections 2.1–2.2), their edge matricizations and hierarchical ranks (Section 2.3), the HSVD (Section 2.4) and the relevant notions of rank (Section 2.5). It then shows that deep polynomial networks are weight-tied TTNs and defines hierarchical features and learning capacity (Section 2.6), and constructs the TTN surrogate of a smooth DNN (Section 2.7). Section 5 lists the experiments suggested in the text.

## 2 Theoretical background

We define tree tensor networks and review some of their important mathematical properties. Throughout, vector spaces are finite-dimensional real Euclidean spaces  $\mathbb{R}^d$  with their standard basis and inner product, and we use the resulting identification  $\mathbb{R}^d \cong (\mathbb{R}^d)^*$  freely (Furman et al., 2026, Appendix A). General references are Kolda and Bader (2009), Hackbusch (2012) and Lim (2021). We write  $[n] = \{1, \dots, n\}$ .

## 2.1 Tensors, multilinear maps and matricizations

**Definition 2.1** (Tensor). Let  $d_1, \dots, d_n$  be positive integers. An *order- $n$  tensor* is an element  $T \in \mathbb{R}^{d_1} \otimes \dots \otimes \mathbb{R}^{d_n}$ . Expanding  $T = \sum T_{i_1 \dots i_n} e_{i_1} \otimes \dots \otimes e_{i_n}$  in the standard bases identifies  $T$  with the array  $(T_{i_1 \dots i_n}) \in \mathbb{R}^{d_1 \times \dots \times d_n}$ . The index  $i_k$ , and the space  $\mathbb{R}^{d_k}$ , is the  $k$ -th *mode*, or *leg*, of  $T$ . We write  $\langle S, T \rangle = \sum_{i_1, \dots, i_n} S_{i_1 \dots i_n} T_{i_1 \dots i_n}$  and  $\|T\|_F = \langle T, T \rangle^{1/2}$ .

**Definition 2.2** (Multilinear map). Let  $V_1, \dots, V_p, W$  be vector spaces. A map  $F : V_1 \times \dots \times V_p \rightarrow W$  is  *$p$ -linear* if it is linear in each argument when the others are fixed. By the universal property of the tensor product,  $p$ -linear maps  $V_1 \times \dots \times V_p \rightarrow W$  are in one-to-one correspondence with linear maps  $V_1 \otimes \dots \otimes V_p \rightarrow W$ .

**Proposition 2.3** (Tensors as multilinear maps; Furman et al., 2026, Proposition A.1). Let  $T \in \bigotimes_{k=1}^n \mathbb{R}^{d_k}$ , and let  $I \sqcup O = [n]$  be a partition of the modes into inputs and outputs. The formula

$$F_T((v_k)_{k \in I})_{(i_o)_{o \in O}} = \sum_{(i_k)_{k \in I}} T_{i_1 \dots i_n} \prod_{k \in I} (v_k)_{i_k} \quad (2.1)$$

defines a multilinear map  $F_T : \prod_{k \in I} \mathbb{R}^{d_k} \rightarrow \bigotimes_{o \in O} \mathbb{R}^{d_o}$ , and  $T \mapsto F_T$  is a linear isomorphism onto the space of such maps. Equivalently,  $T$  is a linear map from  $\bigotimes_{k \in I} \mathbb{R}^{d_k}$  to  $\bigotimes_{o \in O} \mathbb{R}^{d_o}$ .

This is the precise sense in which “a tensor is a multilinear map”: it maps a tensor product of vector spaces linearly (equivalently, a Cartesian product of vector spaces multilinearly) to another tensor product of vector spaces. The split of the modes into inputs and outputs is a choice, not a property of  $T$ .

**Example 2.4.** An order-3 tensor  $W \in \mathbb{R}^{d_1} \otimes \mathbb{R}^{d_2} \otimes \mathbb{R}^{d_3}$  with inputs  $\{1, 2\}$  and output  $\{3\}$  is the bilinear map  $B_W(u, v)_k = \sum_{i,j} W_{ijk} u_i v_j$ : a vertex with two input legs and one output leg. With input  $\{1\}$  and outputs  $\{2, 3\}$ , the same  $W$  is a linear map  $\mathbb{R}^{d_1} \rightarrow \mathbb{R}^{d_2} \otimes \mathbb{R}^{d_3}$ . For  $d_1 = d_2 = d_3 = d$ , the diagonal tensor  $\Delta_{ijk} = \mathbf{1}[i = j = k]$  gives the Hadamard (coordinatewise) product  $\Delta(u, v) = u \odot v$ , which is linear in  $u$  and in  $v$  but not jointly linear.

**Definition 2.5** (Matricization). Let  $T$  be an order- $n$  tensor,  $\emptyset \neq A \subsetneq [n]$  with complement  $A^c$ , and  $m_A = \prod_{k \in A} d_k$ . Bipartition the indices of  $T$  into the groups  $A$  and  $A^c$  and flatten each group, using the lexicographic order on multi-indices. The result is the  *$A$ -matricization*  $T^{(A)} \in \mathbb{R}^{m_A \times m_{A^c}}$ , with entries  $(T^{(A)})_{(i_k)_{k \in A}, (i_k)_{k \in A^c}} = T_{i_1 \dots i_n}$ . As a linear map,  $T^{(A)}$  is  $F_T$  with inputs  $A^c$  and outputs  $A$ . Its rank  $\text{rank}_A(T) := \text{rank } T^{(A)}$  does not depend on the orderings, and  $\text{rank}_A(T) = \text{rank}_{A^c}(T)$ . For  $A = \{k\}$  one obtains the mode- $k$  matricization  $T_{(k)}$  (Kolda and Bader, 2009).

## 2.2 Tensor networks and tree tensor networks

**Definition 2.6** (Tensor network). A *tensor network* consists of

- (i) a finite graph with vertex set  $V$  and edge set  $E = E_{\text{int}} \sqcup E_{\text{ext}}$ , where each *internal edge* or *bond*  $e \in E_{\text{int}}$  joins two distinct vertices, and each *external edge* or *open leg*  $a \in E_{\text{ext}}$  is attached to a single vertex;
- (ii) a dimension for each edge: a *bond dimension*  $D_e$  for  $e \in E_{\text{int}}$ , and a *leg dimension*  $d_a$  for  $a \in E_{\text{ext}}$ ;
- (iii) for each vertex  $v$ , a *core*  $C_v$ : a tensor with one mode for each edge incident to  $v$ , of the corresponding dimension.

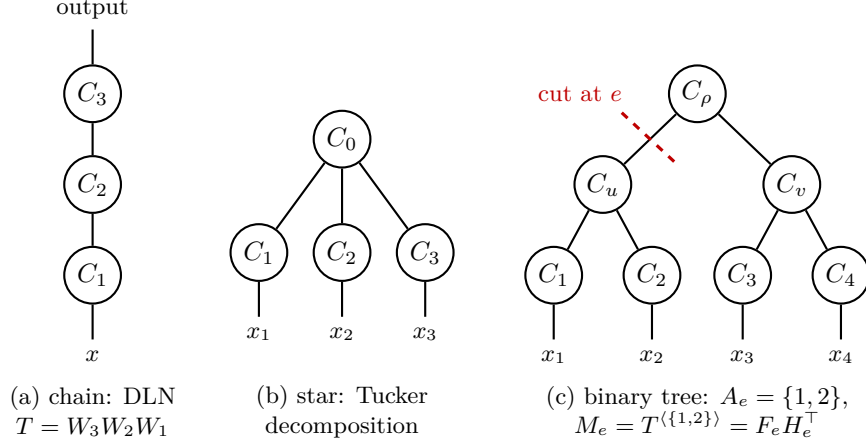


Figure 1: Examples of tree tensor networks. Circles are cores, lines between circles are bonds, and dangling lines are open legs. (a) A chain is a deep linear network. (b) A star is a Tucker decomposition. (c) In a tree, cutting the bond  $e$  splits the open legs into  $A_e = \{1, 2\}$  and its complement; the corresponding matricization factors through the bond,  $M_e = F_e H_e^\top$  (Lemma 2.8), so its rank is at most the bond dimension  $D_e$ .

The tensor *represented* by the network is  $T \in \bigotimes_{a \in E_{\text{ext}}} \mathbb{R}^{d_a}$ , obtained by contracting all bonds:

$$T_{(i_a)_{a \in E_{\text{ext}}}} = \sum_{(i_e)_{e \in E_{\text{int}}}} \prod_{v \in V} (C_v)_{(i_e)_{e \ni v}}, \quad (2.2)$$

where each bond index  $i_e$  ranges over  $[D_e]$ , and  $(i_e)_{e \ni v}$  collects the indices of all edges, bonds and legs, incident to  $v$ . We write  $\vartheta = (C_v)_{v \in V}$  for the parameters and  $T(\vartheta)$  for the represented tensor.

Informally: vertices carry tensors and edges are indices. Each core is linear in each of its legs; an open leg is a free index of the represented tensor; and joining two legs by a bond means summing over the shared index, which composes the corresponding multilinear maps (Furman et al., 2026, Proposition A.4). For instance, the product  $AB$  of two matrices is the network with two vertices and one bond, and a vertex with two input legs and one output leg is a bilinear map (Example 2.4). The order in which the sums in (2.2) are performed is irrelevant.

**Definition 2.7** (Tree tensor network). A *tree tensor network* (TTN) is a tensor network whose graph  $(V, E_{\text{int}})$  is a tree. We always assume that every leaf of this tree (a vertex of degree one in  $(V, E_{\text{int}})$ ) carries at least one open leg.

Choosing a root  $\rho \in V$  and orienting every bond towards  $\rho$  turns a TTN into a feedforward computation: by Proposition 2.3, each non-root core  $C_v$  is a multilinear map from its child bonds and its open legs to its parent bond, and  $T$  is computed by composing these maps from the leaves to the root (Furman et al., 2026, Definition 3.1 and Appendix A.3). The root and the orientation are a presentation choice, since (2.2) does not depend on them. Figure 1 shows three examples: a chain is a DLN  $T = W_L \cdots W_1$ , and a star is a Tucker decomposition. A binary tree with one leg per leaf is the hierarchical Tucker format (Hackbusch and Kühn, 2009; Grasedyck, 2010), and a path with one leg per vertex is a tensor train, or matrix product state (Oseledets, 2011). TTNs originate in quantum many-body physics (Shi et al., 2006).

A TTN defines a model of functions once its legs receive inputs. If the input  $x$  splits into blocks  $(x^{(a)})_{a \in E_{\text{ext}}}$  and each leg receives a fixed embedding  $\xi_a(x^{(a)}) \in \mathbb{R}^{d_a}$  of its block (for example

$\xi(z) = (1, z)$ , or a one-hot encoding), the TTN computes

$$g_{\vartheta}(x) = \left\langle T(\vartheta), \bigotimes_{a \in E_{\text{ext}}} \xi_a(x^{(a)}) \right\rangle, \quad (2.3)$$

which is multilinear in the embedded blocks and nonlinear in  $x$  (Stoudenmire and Schwab, 2016). We will meet two situations: *physical* TTNs, whose legs receive distinct blocks of the input, and *computational* TTNs, whose legs all receive copies of the same input (Section 2.6).

### 2.3 Edge cuts, bond dimensions and hierarchical ranks

The key structural property of a tree is that removing any edge disconnects it. Each bond therefore defines a bipartition of the open legs, hence a matricization of the represented tensor.

**Lemma 2.8** (Edge cuts). Let  $T$  be the tensor represented by a TTN and let  $e \in E_{\text{int}}$ . Removing  $e$  splits the tree into two subtrees with vertex sets  $V_e$  and  $V_e^c$ . Let  $A_e \subseteq E_{\text{ext}}$  be the set of open legs attached to  $V_e$ , and  $A_e^c = E_{\text{ext}} \setminus A_e$ .

- (i)  $A_e$  and  $A_e^c$  are nonempty, so  $e$  defines the matricization  $M_e := T^{(A_e)} \in \mathbb{R}^{m_e \times m_e^c}$ , where  $m_e = \prod_{a \in A_e} d_a$  and  $m_e^c = \prod_{a \in A_e^c} d_a$ .
- (ii)  $M_e = F_e H_e^\top$ , where  $F_e \in \mathbb{R}^{m_e \times D_e}$  (resp.  $H_e \in \mathbb{R}^{m_e^c \times D_e}$ ) is the contraction of the cores in  $V_e$  (resp.  $V_e^c$ ) with the index of the bond  $e$  left open.
- (iii)  $\text{rank } M_e \leq D_e$ .

*Proof.* (i) The subtree on  $V_e$  is either a single vertex, which then has degree one in the tree, or it has at least two leaves, at most one of which is the endpoint of  $e$ . In both cases  $V_e$  contains a leaf of the original tree, which carries an open leg. The same argument applies to  $V_e^c$ . (ii) Since  $e$  is the only bond between  $V_e$  and  $V_e^c$ , the sum (2.2) factorizes as a sum over  $i_e \in [D_e]$  of the product of the contraction over  $V_e$  and the contraction over  $V_e^c$ . (iii) follows from (ii).  $\square$

**Definition 2.9** (Bond dimension and hierarchical rank). The dimension  $D_e$  of the index along which the two subtree matrices  $F_e$  and  $H_e$  are contracted is the *bond dimension* of  $e$ . The family

$$(M_e, r_e)_{e \in E_{\text{int}}}, \quad r_e := \text{rank } M_e,$$

collects the edge matricizations of  $T$  and their ranks, and the tuple  $r(T) := (r_e)_{e \in E_{\text{int}}}$  is the *hierarchical (Tucker) rank* of  $T$  with respect to the tree. By Lemma 2.8,  $r_e \leq D_e$ : the rank across a cut is upper bounded by the bond dimension.

The matrices  $M_e$  and ranks  $r_e$  depend only on the represented tensor  $T$  and on which legs lie on each side of each bond, not on the cores. Cutting an open leg  $a \in E_{\text{ext}}$  instead of a bond gives the mode matricization  $T^{\{a\}}$ .

The bond dimensions describe exactly what a TTN can represent.

**Proposition 2.10** (Realizability; Grasedyck and Hackbusch, 2011; Furman et al., 2026, Proposition B.1). Fix a tree with open legs. A tensor  $T$  is represented by a TTN on this tree with bond dimensions  $(D_e)_{e \in E_{\text{int}}}$  if and only if  $r_e(T) \leq D_e$  for every  $e \in E_{\text{int}}$ . In that case one can take  $D_e = r_e(T)$ .

*Sketch.* Necessity is Lemma 2.8(iii). For sufficiency, the column spaces of the matrices  $M_e$  are nested along the tree (Proposition 2.14 below). Choosing a basis of each column space and expressing it in the bases of the children defines cores with  $D_e = r_e$  that represent  $T$  (Furman et al., 2026, Proposition B.1); padding the cores with zeros gives any larger bond dimensions.  $\square$

Hence the model class of a TTN architecture is the set  $\{T : r_e(T) \leq D_e \forall e \in E_{\text{int}}\}$ . Since the set of matrices of rank at most  $r$  is closed, this set is closed, and best approximations within it exist; this is not the case for the CP rank (Section 2.5).

**Remark 2.11** (Why trees: canonical matricizations). In a TTN, every bond  $e$  determines exactly one bipartition  $(A_e \mid A_e^c)$  of the open legs, and its bond dimension bounds the rank across that bipartition. The edge matricizations are therefore canonically attached to the network. This is not the case for tensor networks with cycles. There, removing a single edge does not disconnect the graph; a bipartition of the open legs is separated only by a set  $C$  of edges, the rank across it is bounded by  $\prod_{e \in C} D_e$ , and a given edge belongs to the separating sets of many bipartitions. The correspondence between edges and matricizations is lost, and with it several properties used below: no characterization of the model class by ranks of matricizations such as Proposition 2.10 is available in general, sets of tensors representable with bounded bond dimensions need not be closed (Landsberg et al., 2012), and canonical (minimum-norm) parameterizations are no longer unique up to gauge (Furman et al., 2026, Remark F.7).

The parameterization of a TTN is redundant: an invertible change of basis on a bond can be absorbed by the two adjacent cores.

**Proposition 2.12** (Gauge freedom). Let  $e = \{u, v\} \in E_{\text{int}}$  and  $X \in \text{GL}(D_e)$ . For a core  $C$  incident to  $e$ , let  $C \times_e X$  denote the core obtained by multiplying the mode of  $C$  corresponding to  $e$  by  $X$ , that is  $(C \times_e X)_{\dots j \dots} = \sum_i C_{\dots i \dots} X_{ij}$ . Replacing  $C_u$  by  $C_u \times_e X$  and  $C_v$  by  $C_v \times_e X^{-\top}$  leaves  $T$  unchanged. Consequently the *gauge group*  $\mathcal{G} = \prod_{e \in E_{\text{int}}} \text{GL}(D_e)$  acts on the parameters and preserves  $T$ . Every quantity defined from  $T$  alone, in particular  $M_e$ ,  $r_e$ , and the singular values and singular subspaces of  $M_e$ , is gauge invariant, whereas individual core entries are not. If  $D_e = r_e(T)$  for every  $e$  (a minimal representation), then any two parameterizations of  $T$  with these bond dimensions differ by a unique gauge transformation.

*Proof.* If  $V_e$  contains  $u$ , the substitution maps  $F_e$  to  $F_e X$  and  $H_e$  to  $H_e X^{-\top}$ , so  $M_e = F_e H_e^{\top}$  is unchanged; since  $M_e$  is a rearrangement of  $T$ , so is  $T$ . For the uniqueness statement for minimal representations see Hackbusch (2012) and Uschmajew and Vandereycken (2013).  $\square$

## 2.4 The hierarchical singular value decomposition

Since every bond defines a canonical matricization, we can take its singular value decomposition.

**Definition 2.13** (SVD at a bond; hierarchical SVD). For  $e \in E_{\text{int}}$ , let

$$M_e = U_e S_e V_e^{\top}$$

be a compact SVD:  $U_e \in \mathbb{R}^{m_e \times r_e}$  and  $V_e \in \mathbb{R}^{m_e^c \times r_e}$  have orthonormal columns, and  $S_e = \text{diag}(\sigma_{e,1}, \dots, \sigma_{e,r_e})$  with  $\sigma_{e,1} \geq \dots \geq \sigma_{e,r_e} > 0$ . Reshaping the  $j$ -th columns of  $U_e$  and  $V_e$  gives tensors  $u_{e,j}$  over the legs in  $A_e$  and  $v_{e,j}$  over the legs in  $A_e^c$ , and, up to the order of the legs,

$$T = \sum_{j=1}^{r_e} \sigma_{e,j} u_{e,j} \otimes v_{e,j}. \quad (2.4)$$

The  $\sigma_{e,j}$  are the *hierarchical singular values* of  $T$  at  $e$ , and  $u_{e,j}$  (resp.  $v_{e,j}$ ) its left (resp. right) singular vectors, which describe the subtree on one side of  $e$  (resp. its environment). The *hierarchical SVD* (HSVD) of  $T$  is the family  $(U_e, S_e, V_e)_{e \in E_{\text{int}}}$  of the SVDs across all bonds of the TTN (Grasedyck, 2010).

The HSVD is more than a list of unrelated SVDs: the singular vectors at different bonds are nested and fit together into a TTN.

**Proposition 2.14** (Nestedness; the HSVD is a TTN). Root the tree at a vertex  $\rho$  and, for each bond  $e$ , let  $A_e$  be the set of legs below  $e$ . Let  $u \neq \rho$  be a vertex with parent bond  $e$ , child bonds  $e_1, \dots, e_k$ , and open legs of total dimension  $n_u$  ( $n_u = 1$  if  $u$  has no open leg). Then, after reordering the legs,

$$\text{col}(M_e) \subseteq \mathbb{R}^{n_u} \otimes \text{col}(M_{e_1}) \otimes \dots \otimes \text{col}(M_{e_k}). \quad (2.5)$$

Consequently  $B_u := (I_{n_u} \otimes U_{e_1} \otimes \dots \otimes U_{e_k})^\top U_e$  satisfies  $U_e = (I_{n_u} \otimes U_{e_1} \otimes \dots \otimes U_{e_k}) B_u$ . Set  $B_\rho := (I_{n_\rho} \otimes U_{e_1} \otimes \dots \otimes U_{e_k})^\top \text{vec}(T)$  at the root, where  $e_1, \dots, e_k$  now denote the child bonds of  $\rho$ . The TTN with cores  $(B_v)_{v \in V}$  and bond dimensions  $(r_e)_{e \in E_{\text{int}}}$  represents  $T$ , and in it the contraction of the subtree below each bond  $e$  equals  $U_e$ ; in particular  $F_e = U_e$  has orthonormal columns.

*Proof.* Each column of  $M_e$  is a partial contraction of  $T$  against a fixed vector on the legs outside the subtree of  $e$ . Matricizing such a column with respect to the legs below a child bond  $e_j$  gives columns that are partial contractions of  $T$  against vectors on the legs outside the subtree of  $e_j$ , hence columns of  $M_{e_j}$ . A tensor whose matricizations along every child subtree have their columns in  $\text{col}(M_{e_j})$  lies in the right-hand side of (2.5) (Furman et al., 2026, proof of Proposition B.1). The identity for  $U_e$  follows because  $I \otimes U_{e_1} U_{e_1}^\top \otimes \dots \otimes U_{e_k} U_{e_k}^\top$  is the orthogonal projector onto that right-hand side. The representation statement follows by induction from the leaves, the root step using (2.5) for  $T$  itself.  $\square$

**Remark 2.15** (Canonical and gauge invariant). The HSVD depends only on  $T$  and on the tree, so it is gauge invariant (Proposition 2.12). It is unique up to the usual ambiguities of the SVD: the signs of singular vectors, and rotations within the eigenspaces of  $M_e M_e^\top$  associated with repeated singular values. The singular values, and the singular subspaces of distinct singular values, are therefore canonical, and near a degeneracy one should compare whole singular subspaces rather than individual vectors. The orthonormal TTN of Proposition 2.14 is one canonical gauge. Another is the balanced, or minimum-norm, gauge of Furman et al. (2026, Appendix F), which is unique up to the orthogonal gauge group  $\prod_e \text{O}(D_e)$  and preserved by gradient flow; both carry the same gauge-invariant data.

Truncating the HSVD gives a quasi-optimal compression.

**Proposition 2.16** (Truncation; Grasedyck, 2010). Let  $k_e \leq r_e$  for  $e \in E_{\text{int}}$ , and let  $P_e$  be the orthogonal projector onto the span of the  $k_e$  leading left singular vectors of  $M_e$ , acting on the legs in  $A_e$ . Enumerate the bonds  $e_1, \dots, e_m$  so that every bond comes before the bonds below it. Then  $T_k := P_{e_m} \dots P_{e_1} T$  satisfies  $r_e(T_k) \leq k_e$  for every  $e$ , and

$$\|T - T_k\|_F^2 \leq \sum_{e \in E_{\text{int}}} \sum_{j > k_e} \sigma_{e,j}^2 \leq |E_{\text{int}}| \cdot \min \{ \|T - T'\|_F^2 : r_e(T') \leq k_e \ \forall e \in E_{\text{int}} \}.$$

*Proof.* A projection at a bond below  $e$ , or in a subtree disjoint from the subtree below  $e$ , acts on one side of the cut at  $e$  only, so it does not increase  $r_e$ ; projections at bonds above  $e$  are applied before  $P_e$ . For orthogonal projectors  $\pi_1, \dots, \pi_m$ , the decomposition  $A - \pi_m B = (I - \pi_m)A + \pi_m(A - B)$  into orthogonal terms gives, by induction,  $\|A - \pi_m \dots \pi_1 A\|^2 \leq \sum_i \|(I - \pi_i)A\|^2$ , and  $\|(I - P_e)T\|_F^2 = \sum_{j > k_e} \sigma_{e,j}^2$  by the Eckart–Young theorem (Eckart and Young, 1936). Finally, for every  $T'$  with  $r_e(T') \leq k_e$ , Eckart–Young gives  $\sum_{j > k_e} \sigma_{e,j}^2 \leq \|M_e - T'^{(A_e)}\|_F^2 = \|T - T'\|_F^2$ . The minimum exists because the constraint set is closed.  $\square$

If two bonds define the same bipartition, as the two child bonds of a binary root do, projecting at one of them suffices; this gives the factor  $2d - 3$  of Grasedyck (2010) for binary trees with  $d$  leaves.

**Remark 2.17** (Stability). By Weyl’s inequality,  $|\sigma_{e,j}(T) - \sigma_{e,j}(T')| \leq \|M_e(T) - M_e(T')\|_{\text{op}} \leq \|T - T'\|_F$ : hierarchical singular values are 1-Lipschitz functions of the tensor. Singular subspaces are stable only relative to the gap that separates their singular values from the rest of the spectrum (Davis and Kahan, 1970; Wedin, 1972). These facts matter whenever the HSVDs of two models, or of one model at two times, are compared.

## 2.5 Notions of rank

Tensors admit several notions of rank, which coincide for matrices but not for higher-order tensors (Kolda and Bader, 2009; Lim, 2021).

**Definition 2.18** (CP rank). The *CP rank* of  $T$ ,  $\text{rank}_{\text{CP}}(T)$ , is the smallest number  $R$  of rank-one tensors into which  $T$  can be decomposed:  $T = \sum_{s=1}^R a_s^{(1)} \otimes \cdots \otimes a_s^{(n)}$  (Hitchcock, 1927).

The CP rank is challenging to compute in practice: deciding whether  $\text{rank}_{\text{CP}}(T) \leq R$  is NP-hard (Håstad, 1990; Hillar and Lim, 2013). Moreover, for  $n \geq 3$  and  $R \geq 2$  the set of tensors of CP rank at most  $R$  is in general not closed, so a best approximation of low CP rank may fail to exist (de Silva and Lim, 2008).

**Definition 2.19** (Mode rank and Tucker rank). The *mode- $k$  rank* of  $T$  is  $r_k(T) = \text{rank } T_{(k)}$ , the rank of the matricization obtained by separating index  $k$  from the others. The *Tucker* (or multilinear) *rank* is the tuple  $(r_1(T), \dots, r_n(T))$  of all mode ranks. It is the smallest core size in a Tucker decomposition  $T = \sum_{j_1, \dots, j_n} \Omega_{j_1 \dots j_n} u_{j_1}^{(1)} \otimes \cdots \otimes u_{j_n}^{(n)}$  (Tucker, 1966; De Lathauwer et al., 2000), i.e. a TTN on a star (Figure 1b).

For a TTN, the *hierarchical Tucker rank* is the tuple  $(r_e)_{e \in E_{\text{int}}}$  of Definition 2.9 (Hackbusch and Kühn, 2009; Grasedyck, 2010). Mode ranks and hierarchical ranks are ranks of matrices: unlike the CP rank, they can be computed by SVDs, up to a numerical tolerance.

**Proposition 2.20** (Comparison of ranks).

- (i) For every bipartition  $A$ ,  $\text{rank}_A(T) \leq \text{rank}_{\text{CP}}(T)$ . In particular  $r_k(T) \leq \text{rank}_{\text{CP}}(T)$ , and  $r_e(T) \leq \text{rank}_{\text{CP}}(T)$  for every tree.
- (ii) In a rooted TTN, if a vertex  $u$  has parent bond  $e$ , child bonds  $e_1, \dots, e_k$  and open legs of total dimension  $n_u$ , then  $r_e \leq n_u \prod_{j=1}^k r_{e_j}$ .
- (iii) A tensor of CP rank  $R$  is represented on every tree, with every assignment of legs to vertices, by a TTN whose bond dimensions all equal  $R$  and whose cores are diagonal in the bond indices.

*Proof.* (i) The  $A$ -matricization of a rank-one tensor has rank one, and the matrix rank is sub-additive. (ii) Take dimensions in (2.5). (iii) Given  $T = \sum_{s=1}^R \bigotimes_{a \in E_{\text{ext}}} w_s^{(a)}$ , set  $(C_v)_{(i_a)_{a \in \text{legs}(v)}, (j_e)_{e \in \text{bonds}(v)}} = \sum_{s=1}^R \prod_{e \ni v} \mathbf{1}[j_e = s] \prod_a (w_s^{(a)})_{i_a}$ , where  $a$  ranges over the open legs at  $v$  and  $e$  over the bonds at  $v$ . Contracting forces all bond indices to agree, because the tree is connected (Diagonal TTN note, 2026, Section 2.4).  $\square$

The converse of (i) fails dramatically: TTNs with polynomially many parameters represent tensors whose CP rank is exponential in the number of leaves, for almost every choice of cores (Cohen et al., 2016). This is one precise sense in which hierarchical (deep) factorizations are more expressive than shallow ones.

**Remark 2.21** (Ranks and critical points). [Furman et al. \(2026\)](#) use a parameter-level notion:  $\vartheta$  has *full Tucker rank* if, for every bond  $e$  and each of its endpoints  $v$ , the matricization of  $C_v$  that separates the mode of  $e$  has rank  $D_e$ . In general  $r_e(T(\vartheta))$  is at most the ranks of these two matricizations, and equality holds at balanced (minimum-norm) parameters ([Furman et al., 2026](#), Proposition D.2). For a TTN with free cores trained on the Frobenius loss  $\frac{1}{2}\|T(\vartheta) - T^*\|_F^2$  (their setting, in which inputs are drawn uniformly among the entries of  $T$ , i.e. one-hot inputs) towards a realizable target  $T^*$ , every critical point that is not a global minimum is rank deficient ([Furman et al., 2026](#), Theorem 5.2); at such a point the network behaves as a TTN with smaller bond dimensions. For the parity of  $n$  bits ( $n \geq 4$  a power of two), realized on a balanced binary tree with bond dimension 2, the rank-one point whose cores all have entries  $\frac{1}{2}$ , which is a best rank-one approximation, is a saddle of order at least  $n/2$  ([Furman et al., 2026](#), Proposition 5.3). Hierarchical ranks are therefore natural dynamical variables, and saddle-to-saddle dynamics should appear as successive increases of the rank profile. Whether these results extend to other losses, data distributions or constrained cores is open.

## 2.6 Deep polynomial networks as tree tensor networks

We now show that deep networks with polynomial activations are TTNs with tied weights, evaluated on copies of the input.

**Definition 2.22** (Deep polynomial network). A feedforward network with  $L$  hidden layers of widths  $d_1, \dots, d_L$ , acting on inputs  $x \in \mathbb{R}^{d_0}$ , is

$$h_0 = x, \quad z_\ell = W_\ell h_{\ell-1} + b_\ell, \quad h_\ell = \sigma_\ell(z_\ell) \quad (1 \leq \ell \leq L), \quad f_\theta(x) = c^\top h_L + b_{\text{out}}, \quad (2.6)$$

with  $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$ ,  $b_\ell \in \mathbb{R}^{d_\ell}$ ,  $c \in \mathbb{R}^{d_L}$  and  $b_{\text{out}} \in \mathbb{R}$  collected in  $\theta$ , and activations acting coordinatewise,  $\sigma_\ell(z)_i = \sigma_{\ell,i}(z_i)$ . It is a *deep polynomial network* (DPN) if every  $\sigma_{\ell,i}$  is a polynomial of degree at most  $K$ . It is a *deep monomial network* (DMN) if moreover  $\sigma_{\ell,i}(z) = z^K$  and all biases vanish ( $b_\ell = 0$ ,  $b_{\text{out}} = 0$ ). A DMN with  $K = 2$  is a *deep quadratic network* (DQN).

Allowing the polynomial to depend on the neuron covers the Taylor surrogates of Section 2.7. Since degrees multiply under composition, a DPN is a polynomial in  $x$  of degree at most  $K^L$ .

### 2.6.1 Monomial activations: tied TTNs on copies of the input

**Proposition 2.23** (A monomial layer is one TTN vertex). Let  $\sigma(z) = z^K$  and  $W \in \mathbb{R}^{d' \times d}$ . Then  $\sigma(Wh) = A(h, \dots, h)$ , where  $A$  is the symmetric  $K$ -linear map

$$A_{i j_1 \dots j_K} = \prod_{s=1}^K W_{i j_s}, \quad \text{i.e.} \quad A(u_1, \dots, u_K) = (Wu_1) \odot \dots \odot (Wu_K).$$

In particular, for  $K = 2$ ,

$$\sigma(Wx)_i = \left( \sum_j W_{ij} x_j \right)^2 = \sum_{j,k} W_{ij} x_j W_{ik} x_k = A(x, x)_i. \quad (2.7)$$

The proof is the expansion of the product. Graphically, (2.7) is a vertex with one output leg  $i$  and two input legs; equivalently, it is the Hadamard vertex  $\Delta$  of Example 2.4 whose two input legs carry the same (tied) weight matrix  $W$ , applied to two copies of  $x$ . Composing layers gives a tree.

**Proposition 2.24** (DMNs are tied TTNs). Let  $f_\theta$  be a DMN with exponent  $K \geq 2$  and  $L$  layers, and let  $N = K^L$ . Let  $\mathcal{T}_{K,L}$  be the complete  $K$ -ary rooted tree of depth  $L$  in which every non-root vertex at height  $\ell$  carries the core  $A_\ell$  of Proposition 2.23 built from  $W_\ell$ , the root (height  $L$ ) carries the  $K$ -linear form  $c^\top A_L$ , and the vertices at height 1 carry the  $N$  open legs. Let  $T(\theta) \in (\mathbb{R}^{d_0})^{\otimes N}$  be the represented tensor. Then

$$f_\theta(x) = \langle T(\theta), x^{\otimes N} \rangle \quad \text{for all } x \in \mathbb{R}^{d_0},$$

and the bonds between heights  $\ell$  and  $\ell + 1$  have dimension  $d_\ell$ .

*Proof.* Let  $\Phi_\ell : (\mathbb{R}^{d_0})^{K^\ell} \rightarrow \mathbb{R}^{d_\ell}$  be the contraction of a subtree of height  $\ell$ :  $\Phi_0 = \text{id}$  and  $\Phi_\ell(y_1, \dots, y_{K^\ell}) = A_\ell(\Phi_{\ell-1}(y^{(1)}), \dots, \Phi_{\ell-1}(y^{(K)}))$ , where  $y^{(s)}$  is the  $s$ -th block of  $K^{\ell-1}$  consecutive arguments. Since  $A_\ell(h, \dots, h) = \sigma(W_\ell h)$ , induction gives  $\Phi_\ell(x, \dots, x) = h_\ell(x)$ , hence  $f_\theta(x) = c^\top \Phi_L(x, \dots, x) = \langle T(\theta), x^{\otimes N} \rangle$ .  $\square$

**Remark 2.25** (Copies, not copying). The map  $x \mapsto x^{\otimes K}$  is not linear, so no tensor implements “copying” a vector. A DMN is therefore not a tensor network applied to  $x$ : it is the restriction of the  $N$ -linear map  $T(\theta)$  to the diagonal  $(x, \dots, x)$ . The open legs are formal copies of the same input, and a cut of  $\mathcal{T}_{K,L}$  separates copies, not input variables. The same identification, for networks with bilinear layers, is described as a TTN with a copying operation by [Pearce et al. \(2024\)](#) and [Dooms et al. \(2025\)](#) (see [Furman et al., 2026](#), Section 2).

**Remark 2.26** (Hidden-unit symmetries are gauge transformations). Permuting the hidden units of layer  $\ell$  by a permutation matrix  $P$  ( $W_\ell \mapsto PW_\ell$ ,  $W_{\ell+1} \mapsto W_{\ell+1}P^\top$ ) does not change  $f_\theta$ , and neither does rescaling them, for monomial activations, by an invertible diagonal matrix  $\Lambda$  ( $W_\ell \mapsto \Lambda W_\ell$ ,  $W_{\ell+1} \mapsto W_{\ell+1}\Lambda^{-K}$ , with  $c$  in place of  $W_{L+1}$ ). On the tied TTN these symmetries act as gauge transformations, by  $P$  and by  $\Lambda^K$  respectively, on every bond of height  $\ell$  (Proposition 2.12). Hence  $T(\theta)$  and its HSVD are invariant under them, while the weights are not. This is the advantage of the TTN over the DMN parameterization: the HSVD gives a more canonical, gauge-independent description of internal structure. The idea is then to use the TTN representing a DMN to interpret its internal structure, reading the HSVD as the features that the network learns. Section 2.6.3 makes this precise, and Remark 2.39 discusses its limits.

## 2.6.2 General polynomial activations: homogenization

Biases and lower-order terms break homogeneity. They are absorbed by adding a constant coordinate.

**Definition 2.27** (Homogenization). Let  $p(z) = \sum_{k=0}^K a_k z^k$  be a polynomial of degree at most  $K$ . Its degree- $K$  homogenization is

$$p_h(z_0, z) := \sum_{k=0}^K a_k z_0^{K-k} z^k,$$

which equals  $z_0^K p(z/z_0)$  for  $z_0 \neq 0$ . It is homogeneous of degree  $K$ , and  $p_h(1, z) = p(z)$ . The construction generalizes to multivariate polynomial maps: if  $P : \mathbb{R}^n \rightarrow \mathbb{R}^m$  has degree at most  $K$ ,  $P(z) = \sum_{|\alpha| \leq K} c_\alpha z^\alpha$  with  $c_\alpha \in \mathbb{R}^m$ , then  $P_h(z_0, z) = \sum_{|\alpha| \leq K} c_\alpha z_0^{K-|\alpha|} z^\alpha$  is a homogeneous polynomial map on  $\mathbb{R}^{1+n}$  with  $P_h(1, z) = P(z)$ .

A homogeneous polynomial can in turn be identified with a symmetric multilinear map.

**Lemma 2.28** (Polarization). Let  $Q : V \rightarrow W$  be a homogeneous polynomial map of degree  $K$  between finite-dimensional real vector spaces. There is a unique symmetric  $K$ -linear map  $S_Q : V^K \rightarrow W$  such that  $S_Q(y, \dots, y) = Q(y)$  for all  $y$ , namely

$$S_Q(y_1, \dots, y_K) = \frac{1}{K!} \sum_{\emptyset \neq J \subseteq [K]} (-1)^{K-|J|} Q\left(\sum_{j \in J} y_j\right).$$

Conversely, every symmetric  $K$ -linear map  $S$  defines the homogeneous polynomial  $y \mapsto S(y, \dots, y)$ .

This is the classical polarization identity. Combining the two steps, a polynomial of degree at most  $K$  is a homogeneous polynomial of degree  $K$  in one more variable, which is a symmetric  $K$ -linear map on the augmented space. For a polynomial activation, this map is a vertex with one output leg and  $K$  input legs, which is why a DPN has a tree structure.

We write  $H_\ell := \mathbb{R} \oplus \mathbb{R}^{d_\ell} \cong \mathbb{R}^{1+d_\ell}$ , with coordinates indexed from 0, and  $\bar{h} := (1, h) \in H_\ell$  for  $h \in \mathbb{R}^{d_\ell}$ .

**Proposition 2.29** (A polynomial layer is one TTN vertex; [Local TN note, 2026](#), Section 3). Let  $P_\ell(h) = \sigma_\ell(W_\ell h + b_\ell)$  with polynomial activations  $\sigma_\ell(z) = \sum_{s=0}^K \alpha_{\ell,i,s}(z - m_{\ell,i})^s$ , for arbitrary expansion points  $m_{\ell,i}$ . Then  $P_\ell(h) = \sum_{r=0}^K A_\ell^{(r)}[h^{\otimes r}]$ , where  $A_\ell^{(r)} : (\mathbb{R}^{d_\ell-1})^r \rightarrow \mathbb{R}^{d_\ell}$  is the symmetric  $r$ -linear map with components

$$A_{\ell,i}^{(r)} = \sum_{s=r}^K \binom{s}{r} \alpha_{\ell,i,s} q_{\ell,i}^{s-r} w_{\ell,i}^{\otimes r}, \quad q_{\ell,i} = b_{\ell,i} - m_{\ell,i}, \quad (2.8)$$

and  $w_{\ell,i}^\top$  is the  $i$ -th row of  $W_\ell$ . Define  $G_\ell : H_{\ell-1}^K \rightarrow H_\ell$  by, for  $u_s = (a_s, v_s) \in \mathbb{R} \oplus \mathbb{R}^{d_{\ell-1}}$ ,

$$G_\ell(u_1, \dots, u_K)_0 = \prod_{s=1}^K a_s, \quad G_\ell(u_1, \dots, u_K)_{1:d_\ell} = \sum_{r=0}^K \binom{K}{r}^{-1} \sum_{\substack{S \subseteq [K] \\ |S|=r}} A_\ell^{(r)}[v_s : s \in S] \prod_{s \notin S} a_s. \quad (2.9)$$

Then  $G_\ell$  is symmetric and  $K$ -linear, it is the polarization of the homogenized layer map, and

$$G_\ell(\bar{h}, \dots, \bar{h}) = \overline{P_\ell(h)} = (1, P_\ell(h)) \quad \text{for all } h \in \mathbb{R}^{d_{\ell-1}}. \quad (2.10)$$

In coordinates, an entry of  $G_\ell$  with  $r$  nonzero input indices equals the corresponding entry of  $A_\ell^{(r)}$  divided by  $\binom{K}{r}$ , and the only nonzero entry of the constant output slice is  $(G_\ell)_{0;0\dots 0} = 1$ .

*Proof.* Equation (2.8) follows from the binomial expansion of  $(w_{\ell,i}^\top h + q_{\ell,i})^s$ . Each term of (2.9) is linear in each  $u_s$ , and permuting the  $u_s$  permutes the subsets  $S$ , so  $G_\ell$  is symmetric and  $K$ -linear. With  $u_s = \bar{h}$  for all  $s$ , the inner sum consists of  $\binom{K}{r}$  equal terms  $A_\ell^{(r)}[h^{\otimes r}]$ , which gives (2.10). With  $u_s = (a, v)$  for all  $s$ ,  $G_\ell$  returns  $(a^K, \sum_r a^{K-r} A_\ell^{(r)}[v^{\otimes r}])$ , the homogenization of  $h \mapsto (1, P_\ell(h))$ ; by Lemma 2.28,  $G_\ell$  is its polarization.  $\square$

The factors  $\binom{K}{r}^{-1}$  prevent lower-order terms from being counted several times. Equation (2.8) also shows that the cores are constrained: all orders  $r$  reuse the same weight row  $w_{\ell,i}$ , so  $G_\ell$  is not a free tensor.

**Theorem 2.30** (Exact tied-TTN representation of a DPN; [Local TN note, 2026](#), Proposition 1; [Surrogate note, 2026](#), Proposition 1). Let  $f_\theta$  be a DPN with  $L$  layers and activations of degree at most  $K \geq 2$ , and let  $N = K^L$ . Consider the complete  $K$ -ary tree  $\mathcal{T}_{K,L}$  in which every non-root

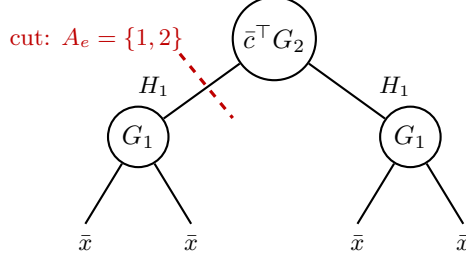


Figure 2: The order-two surrogate of a network with two hidden layers, as a weight-tied binary TTN (Theorem 2.30 and Example 2.32,  $K = L = 2$ ). Both lower vertices carry the same core  $G_1$ ; the  $N = 4$  open legs all receive the augmented input  $\bar{x} = (1, x) \in H_0$ ; the bonds carry augmented hidden states  $\bar{h}_1 \in H_1 = \mathbb{R} \oplus \mathbb{R}^{d_1}$ ; and the root carries  $\bar{c}^\top G_2$ , i.e. its output is contracted with  $\bar{c} = (b_{\text{out}}, c)$ . The cut shown defines  $M_e$  with  $A_e = \{1, 2\}$ ; the other bond of height one gives the complementary bipartition and the same singular values (Proposition 2.33).

vertex at height  $\ell$  carries the core  $G_\ell$  of Proposition 2.29, the root carries the  $K$ -linear form  $\bar{c}^\top G_L$ , i.e. its output is contracted with  $\bar{c} = (b_{\text{out}}, c) \in H_L$ , and the  $N$  open legs, attached to the vertices of height 1, all receive the augmented input  $\bar{x} = (1, x) \in H_0$ . Let  $T(\theta) \in H_0^{\otimes N}$  be the represented tensor. Then

$$f_\theta(x) = \langle T(\theta), \bar{x}^{\otimes N} \rangle \quad \text{for all } x \in \mathbb{R}^{d_0} \text{ and all } \theta, \quad (2.11)$$

and the bonds between heights  $\ell$  and  $\ell + 1$  have dimension  $d_\ell + 1$ . The map  $\theta \mapsto T(\theta)$  is polynomial.

*Proof.* As in the proof of Proposition 2.24, let  $\Phi_\ell : H_0^{K^\ell} \rightarrow H_\ell$  be the contraction of a subtree of height  $\ell$ :  $\Phi_0 = \text{id}$  and  $\Phi_\ell(y_1, \dots, y_{K^\ell}) = G_\ell(\Phi_{\ell-1}(y^{(1)}), \dots, \Phi_{\ell-1}(y^{(K)}))$ . By (2.10) and induction,  $\Phi_\ell(\bar{x}, \dots, \bar{x}) = \bar{h}_\ell(x)$ . Hence  $\langle T(\theta), \bar{x}^{\otimes N} \rangle = \bar{c}^\top \Phi_L(\bar{x}, \dots, \bar{x}) = b_{\text{out}} + c^\top h_L(x) = f_\theta(x)$ . The identity is algebraic, so it holds for every  $\theta$ .  $\square$

So a DPN is a weight-tied TTN applied to copies of the input  $(1, x)$ , the extra coordinate 1 being induced by the homogenization; the bond dimension  $d_\ell + 1$  counts the hidden units of layer  $\ell$  plus the constant. Figure 2 shows the case  $K = L = 2$ . Since (2.11) holds identically in  $\theta$ , it can be differentiated: training the DPN in its own parameters is the same as training the tied TTN through the map  $\theta \mapsto ((G_\ell(\theta))_\ell, \bar{c})$  (Remark 2.52).

**Remark 2.31** (What the representation is, and is not).

- (i) It is a *constrained* TTN: the cores are tied across each level, their entries satisfy the algebraic relations (2.8), and the constant slice is fixed.
- (ii) It is evaluated on the *diagonal*: the leaves are formal copies of the whole input (Remark 2.25).
- (iii) The  $N = K^L$  leaves are never materialized in computations: the recurrence (2.6) evaluates the same function at a cost linear in the depth, by sharing repeated subcomputations, i.e. as a circuit (a directed acyclic graph) rather than a tree.
- (iv) For  $K > 2$ , every  $K$ -ary vertex can be refined exactly into binary vertices, with new bond dimensions given by matricization ranks of  $G_\ell$ .
- (v) The multilinear extension is not unique. Since  $\langle T, \bar{x}^{\otimes N} \rangle = \langle \text{Sym } T, \bar{x}^{\otimes N} \rangle$ , where  $\text{Sym}$  averages over permutations of the  $N$  legs,  $f_\theta$  determines only the symmetric part  $\text{Sym } T(\theta)$ . It determines it uniquely: a homogeneous polynomial on  $H_0$  is determined by its values on the hyperplane  $\{y_0 = 1\}$ , and a symmetric tensor by its homogeneous polynomial (Lemma 2.28). The tensor  $T(\theta)$  is the particular extension induced by the architecture.

**Example 2.32** (Two layers, quadratic activations; [Local TN note, 2026](#), Section 4). For  $K = 2$ , write each layer polynomial as  $P_\ell(h) = a_\ell + A_\ell h + B_\ell(h, h)$  with  $a_\ell \in \mathbb{R}^{d_\ell}$ ,  $A_\ell$  linear and  $B_\ell$  symmetric bilinear (these are  $A_\ell^{(0)}, A_\ell^{(1)}, A_\ell^{(2)}$  in Proposition 2.29). For the order-two Taylor polynomial of  $\sigma_\ell$  at  $m_\ell$  (Section 2.7), with  $\gamma_\ell^{(r)} = \sigma_\ell^{(r)}(m_\ell)$  taken coordinatewise and  $q_\ell = b_\ell - m_\ell$ ,

$$a_\ell = \gamma_\ell^{(0)} + \gamma_\ell^{(1)} \odot q_\ell + \frac{1}{2} \gamma_\ell^{(2)} \odot q_\ell \odot q_\ell, \quad A_\ell u = \gamma_\ell^{(1)} \odot W_\ell u + \gamma_\ell^{(2)} \odot q_\ell \odot W_\ell u, \\ B_\ell(u, v) = \frac{1}{2} \gamma_\ell^{(2)} \odot W_\ell u \odot W_\ell v.$$

The core (2.9) is  $G_\ell((\alpha, u), (\beta, v)) = (\alpha\beta, a_\ell \alpha\beta + \frac{1}{2} A_\ell(\alpha v + \beta u) + B_\ell(u, v))$ . For an output coordinate  $i \geq 1$ , its matrix in the two input indices is

$$(G_\ell)_{i;\dots} = \begin{pmatrix} (a_\ell)_i & \frac{1}{2}(A_\ell)_{i,:} \\ \frac{1}{2}(A_\ell)_{i,:}^\top & (B_\ell)_{i;:, :} \end{pmatrix}.$$

The network is  $f_\theta(x) = \bar{c}^\top G_2(G_1(\bar{x}, \bar{x}), G_1(\bar{x}, \bar{x}))$ , a polynomial of degree four (not two) in  $x$ , with coefficient tensor  $T_{j_1 j_2 j_3 j_4} = \sum_{r,a,b} \bar{c}_r (G_2)_{r;ab} (G_1)_{a;j_1 j_2} (G_1)_{b;j_3 j_4}$ . As a scalar check ( $d_0 = d_1 = d_2 = 1$ ), for the SiLU activation  $\sigma(z) = z/(1 + e^{-z})$  with unit weights, zero biases, centres 0 and unit readout,  $p(z) = z/2 + z^2/4$  and  $f(x) = p(p(x)) = x/4 + 3x^2/16 + x^3/16 + x^4/64$ .

### 2.6.3 Hierarchical features and learning capacity

The representation allows us to define a notion of hierarchical features of a DPN, from the HSVD of its TTN, and a measure of learning capacity, from its hierarchical ranks. For the tied tree, the HSVD reduces to one SVD per layer.

**Proposition 2.33** (One SVD per layer). In the setting of Theorem 2.30,  $T(\theta)$  is invariant under the permutations of its  $N$  legs induced by automorphisms of  $\mathcal{T}_{K,L}$ . Consequently, for two bonds  $e, e'$  at the same height,  $M_{e'}$  is obtained from  $M_e$  by permuting rows and columns: the two bonds have the same hierarchical singular values, and their singular vectors correspond under the relabelling of the leaves.

*Proof.* The cores  $G_\ell$  and  $\bar{c}^\top G_L$  are symmetric in their  $K$  inputs, so the contraction is invariant under exchanging the children of any vertex. These exchanges generate the automorphism group of  $\mathcal{T}_{K,L}$ , which acts transitively on the vertices of each height.  $\square$

We may therefore index the HSVD of a DPN by height: for  $\ell = 1, \dots, L-1$ , it consists of singular values  $\sigma_{\ell,1} \geq \dots \geq \sigma_{\ell,r_\ell} > 0$  and singular vectors  $u_{\ell,j} \in H_0^{\otimes K^\ell}$ ,  $v_{\ell,j} \in H_0^{\otimes(N-K^\ell)}$ . (A network with one hidden layer has no bond; its cuts are mode matricizations.)

**Definition 2.34** (Hierarchical features). The *hierarchical features* of the DPN  $f_\theta$  at height  $\ell$  are the pairs  $(\sigma_{\ell,j}, \varphi_{\ell,j})$ ,  $j \leq r_\ell$ , where

$$\varphi_{\ell,j}(x) := \langle u_{\ell,j}, \bar{x}^{\otimes K^\ell} \rangle$$

is a polynomial of degree at most  $K^\ell$ . The complementary functions are  $\psi_{\ell,j}(x) := \langle v_{\ell,j}, \bar{x}^{\otimes(N-K^\ell)} \rangle$ .

**Proposition 2.35** (What hierarchical features are). For every  $\ell \in \{1, \dots, L-1\}$ :

- (i)  $f_\theta(x) = \sum_{j=1}^{r_\ell} \sigma_{\ell,j} \varphi_{\ell,j}(x) \psi_{\ell,j}(x)$  for all  $x$ ;
- (ii) there is a matrix  $Q_\ell \in \mathbb{R}^{(d_\ell+1) \times r_\ell}$  such that  $\varphi_{\ell,j}(x) = \sum_i (Q_\ell)_{ij} \bar{h}_{\ell,i}(x)$ : the hierarchical features at height  $\ell$  are linear readouts of the augmented activations  $\bar{h}_\ell = (1, h_\ell)$  of layer  $\ell$ ;

(iii)  $r_\ell \leq d_\ell + 1$ .

*Proof.* (i) Contract (2.4) with  $\bar{x}^{\otimes N}$ . (ii) By Lemma 2.8,  $M_e = F_e H_e^\top$ , so  $\text{col}(U_e) = \text{col}(M_e) \subseteq \text{col}(F_e)$ , i.e.  $U_e = F_e Q_\ell$  for some matrix  $Q_\ell$ . The  $i$ -th column of  $F_e$  is the  $i$ -th coordinate of the subtree contraction  $\Phi_\ell$ , and  $\Phi_\ell(\bar{x}, \dots, \bar{x}) = \bar{h}_\ell(x)$  (proof of Theorem 2.30). (iii) is Lemma 2.8(iii).  $\square$

The HSVD at height  $\ell$  thus selects, among the directions of the activation space of layer  $\ell$ , those that the rest of the network uses, ranks them by  $\sigma_{\ell,j}$ , and is invariant under the hidden-unit symmetries of Remark 2.26. Two cautions apply. The orthonormality of the  $u_{\ell,j}$  refers to the Frobenius inner product of coefficient tensors, not to  $L^2(\nu_x)$ . And  $\varphi_{\ell,j}$  and  $\psi_{\ell,j}$  are functions of the same input, so (i) is not a decomposition of  $f_\theta$  into functions of independent variables (Remark 2.39).

**Definition 2.36** (Learning capacity). The *learning capacity* of a DPN at  $\theta$  is the hierarchical rank profile of its TTN,  $r(\theta) = (r_e(T(\theta)))_{e \in E_{\text{int}}}$ , that is  $(r_\ell(\theta))_{\ell=1}^{L-1}$ . It is bounded by the bond dimensions,  $r_\ell \leq D_\ell = d_\ell + 1$ , which form the maximal capacity of the architecture.

By Proposition 2.10, a free TTN with bond dimensions  $D$  represents exactly the tensors with  $r \leq D$ , so  $r(\theta)$  measures how much of the available capacity the model uses. Remark 2.21 further suggests that learning proceeds by increasing ranks, although its results are proved for free TTNs on the Frobenius loss, not for the tied TTN of a DPN, whose loss depends on  $T(\theta)$  only through  $\text{Sym } T(\theta)$  (Remark 2.31(v)). A tuple of ranks raises a natural question.

**Question 2.37** (A global notion of learning capacity). What would be a reasonable global notion of learning capacity? The minimum, the sum, or the maximum of the ranks? Or some other complexity measure using the structure of the TTN?

**Remark 2.38** (Some candidates).

- (a)  $\min_e r_e$  and  $\max_e r_e$  measure the narrowest interface and the cost of contraction, respectively, but ignore where the ranks sit in the tree.
- (b)  $\sum_e \log r_e$  is the logarithm of the product of the ranks;  $\log r_e$  bounds the entropy of the normalized squared singular values  $\sigma_{e,j}^2 / \sum_i \sigma_{e,i}^2$  at  $e$  (the entanglement entropy of quantum physics).
- (c) For a feasible profile  $r$ , the dimension of the set of tensors whose rank profile is exactly  $r$ ,

$$\dim(r) = \sum_{v \in V} n_v \prod_{e \ni v} r_e - \sum_{e \in E_{\text{int}}} r_e^2,$$

where  $n_v$  is the product of the dimensions of the open legs at  $v$ , counts the parameters of a minimal TTN modulo the gauge group. It weights each rank by its position in the tree, and reduces to  $r(m+n-r)$  for  $m \times n$  matrices (Holtz et al., 2012; Uschmajew and Vandereycken, 2013).

- (d) Exact ranks are unstable, since a generic perturbation makes them maximal. Continuous surrogates, such as the participation ratio  $(\sum_j \sigma_{e,j}^2)^2 / \sum_j \sigma_{e,j}^4$  (Diagonal TTN note, 2026, Section 9.4) or the entropy-based effective rank (Roy and Vetterli, 2007), can be aggregated as in (a)–(c).

Which of these, if any, predicts learning behaviour is an empirical question.

**Remark 2.39** (Which tensor carries the features?). Two caveats, emphasized by Local TN note (2026, Section 8) and Surrogate note (2026, Sections 3.4 and 7.4), qualify the identification of the HSVD of  $T(\theta)$  with the features learned by the network.

First, the HSVD of  $T(\theta)$  is invariant under gauge transformations, hence under hidden-unit symmetries, but it is not an invariant of the function  $f_\theta$ . By Remark 2.31(v),  $f_\theta$  determines only

$\text{Sym } T(\theta)$ , and different networks computing the same function can have different hierarchical ranks (Example 2.40). The HSVD is an invariant of the computation, not of the input–output map.

Second, cuts of the unrolled tree separate copies of the input, not input variables. When the input has independent blocks  $x = (x_S, x_{S^c})$ , with  $\nu_x = \mu_S \otimes \mu_{S^c}$ , an architecture-independent alternative is the functional SVD across a physical cut: every  $g \in L^2(\nu_x)$  has a Schmidt decomposition

$$g(x_S, x_{S^c}) = \sum_{q \geq 1} s_q(g) \phi_q^S(x_S) \phi_q^{S^c}(x_{S^c}),$$

with  $(\phi_q^S)_q$  orthonormal in  $L^2(\mu_S)$  and  $(\phi_q^{S^c})_q$  orthonormal in  $L^2(\mu_{S^c})$ . Its singular values and subspaces depend only on  $g$  and  $\nu_x$ , so they can be compared between a teacher, a DNN, its polynomial surrogate and any TTN. Organizing such cuts along a tree of input blocks gives a physical-block TTN and HSVD, whose ranks are not bounded by the widths  $d_\ell$ . In practice, they can be estimated from the crossed-grid matrix  $[g(u_a, v_b) / \sqrt{n_L n_R}]_{a \leq n_L, b \leq n_R}$  built from independent samples  $u_a \sim \mu_S$  and  $v_b \sim \mu_{S^c}$  (Local TN note, 2026, Section 8.4). The computational HSVD of Definition 2.34 and the physical one answer different questions: the first describes how the network computes, the second what it computes.

**Example 2.40** (Same function, different hierarchical ranks). Take  $d_0 = 2$  and two DQNs with two layers (Definition 2.22,  $K = L = 2$ ) computing  $f(x) = x_1^2 x_2^2$ . Network P has  $W_1 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ , so that  $h_1 = ((x_1 + x_2)^2, (x_1 - x_2)^2)$ , then  $W_2 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$  and  $c = 1$ , so that  $z_2 = x_1 x_2$  and  $f = z_2^2$ . Network Q has  $W_1 = I_2$ , so that  $h_1 = (x_1^2, x_2^2)$ , then  $W_2 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$  and  $c = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$ . With the bilinear forms  $Z(u, v) = \frac{1}{2}(u_1 v_2 + u_2 v_1)$ ,  $Z_a(u, v) = u_1 v_1$  and  $Z_b(u, v) = u_2 v_2$ , Proposition 2.24 gives

$$\begin{aligned} T_P(y_1, y_2, y_3, y_4) &= Z(y_1, y_2) Z(y_3, y_4), \\ T_Q(y_1, y_2, y_3, y_4) &= \frac{1}{2} [Z_a(y_1, y_2) Z_b(y_3, y_4) + Z_b(y_1, y_2) Z_a(y_3, y_4)]. \end{aligned}$$

Both restrict to  $x_1^2 x_2^2$  on the diagonal, but the hierarchical rank of the bond at height one is  $r_1 = 1$  for P and  $r_1 = 2$  for Q. By contrast, for independent inputs  $x_1$  and  $x_2$ , the functional rank of  $f$  across the physical cut  $x_1 \mid x_2$  is 1, whichever network computes it.

**Question 2.41** (Interpretability). If we use the TTN as a surrogate model and its HSVD to interpret it, do the hierarchical singular values and vectors correspond to some meaningful structure? For example, if we use a toy model that we can interpret, do the features of the TTN correspond to the known features? Modular arithmetic would be an interesting example to investigate.

The following computation shows what to expect in the modular arithmetic case, in which the mechanism learned by small networks is known.

**Example 2.42** (Modular addition). Let  $p$  be an odd prime, encode inputs  $a, b \in \mathbb{Z}_p$  as one-hot vectors, and let a model output logits  $o(a, b) \in \mathbb{R}^p$ , as in the grokking setting of Power et al. (2022). Since every function of  $(a, b)$  is bilinear in the one-hot encodings, the model is exactly a tensor  $L \in \mathbb{R}^{p \times p \times p}$ ,  $L_{abc} = o(a, b)_c$ : a TTN on the physical tree with leaves  $a, b$  and output  $c$ . Its HSVD across the cut that separates  $a$  is the SVD of  $L^{\langle \{a\} \rangle} \in \mathbb{R}^{p \times p^2}$ . This is a function-level (physical) HSVD: it depends only on the logits, not on the architecture that computes them (Remark 2.39).

- (i) The target  $T_{abc}^* = \mathbf{1}[c \equiv a + b]$  has a maximally degenerate spectrum. The rows of  $T^{\star \langle \{a\} \rangle}$  are orthogonal and each contains  $p$  ones, so  $T^{\star \langle \{a\} \rangle} (T^{\star \langle \{a\} \rangle})^\top = p I_p$  and all  $p$  singular values equal  $\sqrt{p}$ . The SVD alone therefore singles out no basis, in particular not the Fourier basis. The same holds for the two other cuts.

- (ii) [Nanda et al. \(2023\)](#) found that one-layer transformers trained on modular addition implement a Fourier algorithm whose logits are approximately  $L_{abc} = \sum_{k \in \mathcal{F}} \lambda_k \cos(2\pi k(a+b-c)/p)$ , for a small set  $\mathcal{F} \subseteq \{1, \dots, (p-1)/2\}$  of key frequencies. If the  $\lambda_k > 0$  are distinct, then  $L^{\langle \{a\} \rangle}$  has rank  $2|\mathcal{F}|$  and singular values  $\lambda_k p^{3/2}/2$ , each with multiplicity two, and the corresponding left singular subspaces are  $\text{span}\{(\cos(2\pi ka/p))_a, (\sin(2\pi ka/p))_a\}$ . Indeed, with  $\omega_k = 2\pi k/p$ ,

$$\cos(\omega_k(a+b-c)) = \cos(\omega_k a) \cos(\omega_k(b-c)) - \sin(\omega_k a) \sin(\omega_k(b-c)),$$

and these vectors are mutually orthogonal across  $k$  and between  $\cos$  and  $\sin$ , with squared norms  $p/2$  as functions of  $a$  and  $p^2/2$  as functions of  $(b, c)$ .

For this idealized mechanism, the physical HSVD thus recovers exactly the known features, as two-dimensional frequency subspaces ranked by their strengths  $\lambda_k$ ; the rotation within each subspace reflects the translation symmetry of  $\mathbb{Z}_p$ . Question 2.41 asks more: whether the HSVD of the computational TTN of a trained network (Section 2.6.3), which describes how the network computes, also recovers these subspaces. A natural test case is an MLP with quadratic activations acting on the concatenated one-hot inputs, for which an analytic Fourier solution is known in the case of one hidden layer ([Gromov, 2023](#)). With one hidden layer, the computational cuts are the mode matricizations of the root core; with more layers, Definition 2.34 applies.

## 2.7 Surrogate model of a deep neural network

Consider a general DNN (2.6) with smooth activation functions, more precisely of class  $C^{k+1}$  on intervals that contain all the pre-activations of the network and of its surrogate. The idea is to Taylor expand the DNN into a DPN around a training checkpoint, and to represent this DPN as a TTN.

**Definition 2.43** (Layerwise Taylor surrogate; [Local TN note, 2026](#), Section 2.2). Let  $\theta_j = \theta(t_j)$  be a checkpoint,  $\mathcal{C}$  a finite calibration set of inputs, and  $k \geq 1$ . For every layer  $\ell$  and neuron  $i$ , let the *centre*

$$m_{\ell,j,i} := \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} z_{\ell,i}(\theta_j, x)$$

be the average pre-activation of the neuron over the calibration set at the checkpoint, and let

$$p_{\ell,j,i}^{(k)}(z) := \sum_{r=0}^k \frac{\sigma_{\ell,i}^{(r)}(m_{\ell,j,i})}{r!} (z - m_{\ell,j,i})^r$$

be the order- $k$  Taylor polynomial of  $\sigma_{\ell,i}$  at that centre. The *order- $k$  surrogate at checkpoint  $j$*  is the DPN obtained by replacing every activation by its Taylor polynomial, while using the surrogate's own hidden states:

$$\hat{h}_0 = x, \quad \hat{z}_\ell = W_\ell \hat{h}_{\ell-1} + b_\ell, \quad \hat{h}_\ell = p_{\ell,j}^{(k)}(\hat{z}_\ell), \quad \hat{f}_j(\theta, x) = c^\top \hat{h}_L + b_{\text{out}}.$$

The centres and the Taylor coefficients are frozen, and  $\theta$  remains trainable.

**Definition 2.44** (TTN surrogate). For  $k \geq 2$ , the *TTN surrogate* of the DNN at checkpoint  $j$  is the tied TTN of Theorem 2.30, with  $K = k$ , that represents the surrogate:  $\hat{f}_j(\theta, x) = \langle T_j(\theta), \bar{x}^{\otimes k^L} \rangle$ . We say that this TTN is a surrogate model of the DNN at the training point  $\theta_j$ .

**Remark 2.45** (On the definition).

- (i) The surrogate must use its own hidden states: substituting the true hidden states of the DNN at every layer would define an error diagnostic, not a trainable model.
- (ii) Its input degree is  $k^L$ , not  $k$ . Layerwise Taylorization is not the Taylor expansion of the predictor in  $\theta$  (Bai et al., 2020), since composition keeps cross-layer terms of all orders. For  $k = 1$ , every activation is replaced by its tangent line and the surrogate is a deep affine network, i.e. a DLN with biases, not the linearized (NTK) model; its TTN degenerates into a chain. The family of surrogates therefore starts from the model whose dynamics is understood.
- (iii) The centre lives in pre-activation space, since  $\sigma$  is expanded in its argument. Expanding around the mean pre-activation over a calibration set places the expansion at a typical value over the input distribution; the mean minimizes the mean squared distance to the centre, which controls the order-one remainder, but it is a heuristic and not an optimal choice in general.
- (iv) Choosing instead the per-example centre  $m_{\ell,j}(x) = z_{\ell}(\theta_j, x)$  makes the surrogate agree with the DNN and its  $\theta$ -derivatives up to order  $k$  at  $\theta_j$ , so that  $\hat{f}_j(\theta_j + \Delta, \cdot) - f_{\theta_j + \Delta} = O(\|\Delta\|^{k+1})$  under suitable bounds. Its coefficients, however, depend on  $x$ , so it is not one polynomial in  $x$  and has no single TTN. It is a useful control, which separates the error due to the spread of pre-activations across inputs from the error due to training (Local TN note, 2026, Section 2.3).
- (v) Layerwise Taylorization of trained networks also underlies NN2Poly (Moralá et al., 2025); for higher-order dynamical approximations in parameter space, see Bai et al. (2020) and Huang and Yau (2020).
- (vi) In automatic differentiation, the centres and the derivatives  $\sigma^{(r)}(m)$  must be detached from the computational graph, while the cores  $G_{\ell}$ , recomputed from the current  $\theta$ , must remain differentiable.

We would like to know how good an approximation of the DNN such a TTN is. Since training follows gradients, the relevant errors are on derivatives as well as values.

**Lemma 2.46** (Taylor remainders). If  $\sigma$  is  $C^{k+1}$  on the segment between  $m$  and  $z$ , and  $M_{k+1}$  bounds  $|\sigma^{(k+1)}|$  there, then

$$|\sigma(z) - p^{(k)}(z)| \leq \frac{M_{k+1}}{(k+1)!} |z - m|^{k+1}, \quad |\sigma'(z) - (p^{(k)})'(z)| \leq \frac{M_{k+1}}{k!} |z - m|^k.$$

*Proof.* Use the Lagrange form of the remainder; for the second inequality, note that  $(p^{(k)})'$  is the order- $(k-1)$  Taylor polynomial of  $\sigma'$  at  $m$ .  $\square$

**Proposition 2.47** (Loss-gradient defect; Local TN note, 2026, Proposition 2). Let  $\mathcal{L}(\theta) = \frac{1}{2} \mathbb{E}_{\nu}[(f_{\theta}(x) - y)^2]$  and  $\hat{\mathcal{L}}_j(\theta) = \frac{1}{2} \mathbb{E}_{\nu}[(\hat{f}_j(\theta, x) - y)^2]$ , and define at a common parameter  $\theta$

$$\begin{aligned} \varepsilon_{0,j} &= \|f_{\theta} - \hat{f}_j(\theta, \cdot)\|_{L^2(\nu_x)}, & \varepsilon_{1,j} &= \|\nabla_{\theta} f_{\theta} - \nabla_{\theta} \hat{f}_j(\theta, \cdot)\|_{L^2(\nu_x)}, \\ R_y &= \|f_{\theta} - y\|_{L^2(\nu)}, & B_j &= \|\nabla_{\theta} \hat{f}_j(\theta, \cdot)\|_{L^2(\nu_x)}, \end{aligned}$$

where  $\|v\|_{L^2} = \mathbb{E}[\|v\|^2]^{1/2}$  for vector-valued  $v$ . Whenever differentiation under the expectation is valid,

$$\|\nabla \mathcal{L}(\theta) - \nabla \hat{\mathcal{L}}_j(\theta)\| \leq R_y \varepsilon_{1,j} + B_j \varepsilon_{0,j} =: \beta_j(\theta).$$

*Proof.*  $\nabla \mathcal{L} - \nabla \hat{\mathcal{L}}_j = \mathbb{E}[(f - y)(\nabla f - \nabla \hat{f}_j) + (f - \hat{f}_j)\nabla \hat{f}_j]$ ; apply the triangle and Cauchy–Schwarz inequalities.  $\square$

Two functions can thus be close while their parameter derivatives, and therefore their training updates, are different: prediction agreement alone is not sufficient.

**Proposition 2.48** (Trajectory comparison; [Local TN note, 2026](#), Section 6). Let  $\theta$  and  $\tilde{\theta}$  solve  $\dot{\theta} = -\nabla\mathcal{L}(\theta)$  and  $\dot{\tilde{\theta}} = -\nabla\hat{\mathcal{L}}_j(\tilde{\theta})$  on  $[t_j, t_j + h]$ , and assume that both stay in a set  $\mathcal{U}$  on which  $\nabla\mathcal{L}$  is  $\kappa_j$ -Lipschitz and  $\|\nabla\mathcal{L} - \nabla\hat{\mathcal{L}}_j\| \leq \beta_j$ . Then  $E(s) := \|\theta(t_j + s) - \tilde{\theta}(t_j + s)\|$  satisfies

$$E(s) \leq e^{\kappa_j s} E(0) + \beta_j \frac{e^{\kappa_j s} - 1}{\kappa_j}, \quad 0 \leq s \leq h,$$

the fraction being  $s$  if  $\kappa_j = 0$ . For gradient descent with steps  $\eta_n$ ,  $E_{n+1} \leq (1 + \eta_n \kappa_j) E_n + \eta_n \beta_j$ .

*Proof.* The upper Dini derivative satisfies  $D^+ E \leq \|\nabla\mathcal{L}(\theta) - \nabla\mathcal{L}(\tilde{\theta})\| + \|\nabla\mathcal{L}(\tilde{\theta}) - \nabla\hat{\mathcal{L}}_j(\tilde{\theta})\| \leq \kappa_j E + \beta_j$ ; integrate (Grönwall). The discrete case is identical.  $\square$

These statements make the quality of the approximation precise: the local surrogate is faithful over windows on which the loss-gradient defect  $\beta_j$  is small compared with the gradient itself. They are conditional, and their constants can grow with depth, width and parameter norms; [Local TN note \(2026, Appendix A\)](#) propagates the remainders of Lemma 2.46 through the layers into  $\varepsilon_{0,j}$  and  $\varepsilon_{1,j}$ .

**Remark 2.49** (Fixed centres have an error floor). With input-independent centres, the radius  $|z - m|$  in Lemma 2.46 includes the spread  $S_{\ell,j} = \sup_x \|z_\ell(\theta_j, x) - m_{\ell,j}\|_\infty$  of the pre-activations around the centres, which does not shrink with the time window ([Local TN note, 2026](#), Section 6.3). The gradient defect  $\|\nabla\mathcal{L}(\theta_j) - \nabla\hat{\mathcal{L}}_j(\theta_j)\|$  at the checkpoint is then generally nonzero, and since both trajectories start at  $\theta_j$ ,  $E(s) = s \|\nabla\mathcal{L}(\theta_j) - \nabla\hat{\mathcal{L}}_j(\theta_j)\| + o(s)$ : the error grows linearly from the start of the window, and Proposition 2.48 gives a matching upper bound. If the fixed-centre surrogate fails while the per-example surrogate of Remark 2.45(iv) succeeds, the problem is the spread of activations across inputs, not the tensor representation. Chebyshev or distribution-weighted polynomial fits over the observed range of pre-activations are then natural, non-Taylor alternatives ([Surrogate note, 2026](#), Section 8.2).

We would also like to know whether the HSVD of the TTN can be used to extract the structure that the DPN has learned, and to predict its training dynamics. The following identity connects the two.

**Proposition 2.50** (Dynamics of hierarchical singular values). Let  $N = k^L$ . Along the gradient flow  $\dot{\theta} = -\nabla\hat{\mathcal{L}}_j(\theta)$  of the surrogate, the tensor  $T(t) := T_j(\theta(t))$  evolves as

$$\dot{T} = -\mathcal{K}_\theta[R_\theta], \quad \mathcal{K}_\theta := DT_j(\theta) DT_j(\theta)^*, \quad R_\theta := \mathbb{E}_\nu[(\hat{f}_j(\theta, x) - y) \bar{x}^{\otimes N}],$$

where  $\mathcal{K}_\theta$  is a positive semidefinite operator on  $H_0^{\otimes N}$  and  $R_\theta$  is the residual-weighted moment tensor. If  $\sigma_{e,i}$  is a simple nonzero singular value of  $M_e(t)$ , then it is differentiable and

$$\dot{\sigma}_{e,i} = u_{e,i}^\top \dot{M}_e v_{e,i} = -\langle Y_{e,i}, \mathcal{K}_\theta[R_\theta] \rangle,$$

where  $Y_{e,i}$  is the tensor whose  $A_e$ -matricization is  $u_{e,i} v_{e,i}^\top$ .

*Proof.* We have  $\hat{\mathcal{L}}_j = \mathcal{J} \circ T_j$  with  $\mathcal{J}(T) = \frac{1}{2} \mathbb{E}_\nu[(\langle T, \bar{x}^{\otimes N} \rangle - y)^2]$ , whose gradient is  $R$ . By the chain rule,  $\dot{\theta} = -DT_j(\theta)^*[R_\theta]$  and  $\dot{T} = DT_j(\theta)[\dot{\theta}]$ . For a simple singular value with unit singular vectors,  $\sigma = u^\top M v$  and  $\dot{\sigma} = u^\top \dot{M} v + \sigma(\dot{u}^\top u + v^\top \dot{v}) = u^\top \dot{M} v$ , because  $\|u\| = \|v\| = 1$ .  $\square$

The identity is exact but not closed:  $\mathcal{K}_\theta$ ,  $R_\theta$  and the singular vectors all evolve. Predicting the training dynamics from the HSVD amounts to finding regimes in which a few singular values

and subspaces approximately close under these equations (Surrogate note, 2026, Section 12), as the singular modes of a DLN do in the analysis of Saxe et al. (2014). When surrogates built at different checkpoints are compared, their HSVDs should be evaluated at the same parameter value, since otherwise a change of Taylor centres would be mistaken for learning (Local TN note, 2026, Section 8.5; Surrogate note, 2026, Section 9.1); Remark 2.17 bounds how much the spectrum can move.

**A more direct approach: polynomial activations.** Another, more direct approach is to replace the activation functions of the original network by polynomial activation functions. We then train the corresponding TTN, define some useful observables, such as the HSVD, and check whether their training dynamics is similar to that of the original DNN.

**Definition 2.51** (Polynomial-activation proxy). Given the architecture (2.6) with activation  $\sigma$ , choose polynomials  $q_\ell$  of degree  $K$ , for example the Taylor polynomial of  $\sigma$  at a fixed point, or a least-squares fit of  $\sigma$  on an interval  $[-a, a]$  that contains most pre-activations. The *polynomial proxy*  $f_\theta^q$  is the DPN obtained by replacing  $\sigma_\ell$  with  $q_\ell$ , initialized and trained exactly like the DNN. By Theorem 2.30,  $f_\theta^q(x) = \langle T^q(\theta), \bar{x}^{\otimes K^L} \rangle$ , so the hierarchical singular values  $\sigma_{\ell,j}(t)$  and ranks  $r_\ell(t)$  of Definitions 2.34 and 2.36 can be tracked along its training. They are diagnostics of the TTN representation; they can be compared with the DNN through the HSVDs of its local TTN surrogates (Definition 2.44), or through physical HSVDs (Remark 2.39).

**Remark 2.52** (Algebraic equivalence is not dynamical equivalence). “Training the corresponding TTN” can mean two different things. (a) Training the DPN in its parameters  $\theta$  is identical to training the tied TTN through the map  $\Gamma : \theta \mapsto ((G_\ell(\theta))_\ell, \bar{c})$ , because (2.11) holds identically in  $\theta$ . (b) Training the cores  $g$  as free variables is a different dynamical system: if  $g = \Gamma(\theta)$  and  $\mathcal{J}(g)$  denotes the loss as a function of the cores, (a) induces  $\dot{g} = -D\Gamma(\theta)D\Gamma(\theta)^\top \nabla_g \mathcal{J}(g)$ , a preconditioned and possibly degenerate flow, whereas (b) is  $\dot{g} = -\nabla_g \mathcal{J}(g)$ . Moreover, free training breaks the ties and constraints of Remark 2.31(i). Surrogate note (2026, Section 4.1) proposes a ladder of relaxations that isolates each simplification: the tied TTN trained through  $\theta$ ; free tied cores with the induced metric; a block-diagonal approximation of that metric; Euclidean core gradients; untied cores; compressed ranks; and finally a different tree. For computations, one should use the recurrence (2.6) rather than the exponentially large tree.

**Remark 2.53** (Choosing the degree). A problem is to choose the degree of the nodes of the TTN. Several considerations bear on it.

- (i) Approximation: on a bounded range of pre-activations, least-squares or Chebyshev approximations of a smooth activation improve rapidly with  $K$ , whereas Taylor polynomials are accurate only near their centre.
- (ii) Stability: polynomials are unbounded and the degree  $K^L$  of the network grows exponentially with depth, so pre-activations that leave the fitted range are amplified. The fraction of pre-activations outside that range is a direct diagnostic (Surrogate note, 2026, Section 8.8).
- (iii) Structure:  $K = 2$  is the smallest nonlinear choice and gives binary trees. Networks with bilinear layers  $(Wx) \odot (Vx)$ , which are untied versions of quadratic activations, are competitive in practice and can be interpreted through their tensors (Pearce et al., 2024).
- (iv) Cost: each core has order  $K + 1$ , while the bond dimensions  $d_\ell + 1$  do not depend on  $K$ .

A pragmatic rule is to choose the smallest  $K$  for which the defects of Proposition 2.47 are below a tolerance at initialization and at a few checkpoints.

### 3 Training dynamics of TTNs

### 4 How to compare TTNs and DNNs

We want to test experimentally whether the TTN training dynamics is similar to some DNN with more realistic architecture training dynamics. A first step would be to compare TTN with deep neural networks with standard activation functions (ReLU, GeLU etc). For simplicity we can start with a binary teacher tree tensor network of depth  $L$  generating some training data. The features are defined by the hierarchical SVD of the TTN. The feature of the DNN are defined by a functional SVD of the DNN. Here is how the functional SVD works. For a tree cut at edge  $e$  we matricize the TTN which partitions the inputs into  $I_e = \{x_1, \dots, x_{i_e}\}$  and  $J_e = \{x_{i_e+1}, \dots, x_{d_0}\}$ . For a bipartition of the input into  $x = (x_{I_e}, x_{J_e})$  we construct the matrix  $f_\theta(x_{I_e}, x_{J_e})$  of input-output relations for the function  $f$  of the DNN. We SVD this matrix and for each edge  $e$  this gives a functional HSVD of the DNN which defines the hierarchical features of the DNN. The function writes:

$$f(x_{I_e}, x_{J_e}) \approx \sum_q \sigma_q \phi_q(x_{I_e}) \psi_q(x_{J_e})$$

**Question** If the input dim is very large this could be a big matrix. Is there some principled computational proxy? In practice we form the functional SVD by sampling  $n_L$  inputs  $x_{I_e, i} \sim \mu_L$  and  $n_R$  inputs  $x_{J_e, j} \sim \mu_R$  with data distribution  $\mu \approx \mu_L \otimes \mu_R$  from the sampling distribution and we estimate the average:

$$M_{I_e, J_e} = \frac{1}{\sqrt{n_R n_L}} \sum_{i, j} f(x_{I_e, i}, x_{J_e, j})$$

The SVD of  $M_{I_e, J_e}$  will be the HSVD of the function  $f$  at edge  $e$ . A big limitation about this approach is assuming that we can factorize the data distribution. In general we don't know such factorisation and this is partly why we are using DNNs in the first place. However this could still be used as a ground truth test.

A more scalable approach would be to decode the activations of the DNN and compare them with the features learned by the TTN. At least for the student and teacher TTNs there is a clean notion of hierarchical features being learned that are given by their HSVDs.

For the DNN, consider the activations at layer  $l$  given by  $h_l(x)$  we can define a feature empirically by decoding these representations into a linear probe:

$$D_{l, v} = 1 - \frac{\min_w (\mathbb{E}[(\chi_v - w^\top h_l(x, t))^2])}{\text{var} \chi_v}$$

Note that we need to define the feature  $\chi_v$  of the tree. They are related to the HSVD of the TTN.

Consider the tensor teacher tensor  $T$ . Consider its HSVD:  $(U_e, S_e, V_e)_e$ . Consider the input data distribution  $x$  sampled from some distribution with data covariance matrix  $\Sigma$ . For the edge  $e$  we can write the tensor  $T$  via its matricisation at  $e$ :

$$T = \sum_k S_{e, k} u_e v_e^\top$$

In particular across one output leg, for a bipartition of the inputs legs into  $I_e$  and  $I_e^c$ , we have the following input-output relationship:

$$y = \sum_k S_{e, k} \langle u_{e, k}, x_{I_e} \rangle \langle v_{e, k}, x_{I_e^c} \rangle$$

If the inputs are whitened, define  $\phi_{e,k} = \langle u_{e,k}, x_{I_e} \rangle$  and  $\psi_{e,k} = \langle v_{e,k}, x_{I_e^c} \rangle$ . The hierarchical  $k$ -feature  $e$  is defined by the tuple  $(S_e, \phi_e, \psi_e)$ .

If the inputs are not whitened we have  $\Sigma_e := \mathbb{E}[x_{I_e} x_{I_e}^\top]$ ;  $\Sigma_e^c := \mathbb{E}[x_{I_e^c} x_{I_e^c}^\top]$  and we define the feature vectors as:  $\phi_{e,k} := u_{e,k}^\top \Sigma_e^{-\frac{1}{2}} x_{I_e}$ ;  $\psi_{e,k} := v_{e,k}^\top \Sigma_e^{-\frac{1}{2}} x_{I_e^c}$ . This comes from the weighting of the function by the data covariance (need to explain in more details here). For degenerate singular values instead of considering one particular feature vector, we extend to the whole subspace of feature vectors associated to a particular singular value.

One atom of computation is defined by  $\sigma_e \phi_e \psi_e$  i.e. one mode of the SVD which is part of the TTN decomposition. To estimate whether the DNN has learned a particular mode of the TTN we can estimate it in the hilbert space of the data by computing:

$$\alpha_{e,k} := \frac{c_{e,k}}{\sigma_{e,k}}$$

For each edge cut  $e$  and  $k$  where:

$$c_{e,k} = \langle f, \phi_e \psi_e \rangle = \mathbb{E}[f(\phi_e \psi_e)^\top]$$

A score of 0 means the feature has not been learned and a score close to 1 means that it has. For degenerate block we can extend this to the whole subspace of degenerate features.

To dive more specifically into the internal structure leading to computing that atom, we check whether  $h_l(x)$  already represent the feature  $k$  for edge  $e$ . We compute the following acquisition score:

$$A_{l,e,k}^z = \frac{\mathbb{E}[z_{e,k} h_l^\top]^\top H_l^\dagger \mathbb{E}[z_{e,k} h_l^\top]}{\text{Var}(z_{e,k})}$$

With  $z = \phi, \psi, \phi\psi$  and  $H_l^\dagger$  is the Moore Penrose pseudo inverse of  $\mathbb{E}[h_l h_l^\top]$

Acquisition score close to 1 means  $h_l$  represent the feature vector (or input or output feature vector). A particularly interesting hypothesis is whether early layer represent features that correspond to edges closer to the leaves and later layer to edges closer to the root. Again for degenerate subspace we can extend to the degenerate features.

Estimating whether some feature has been learned and where it has been learned across layers would be particularly useful things to track experimentally to establish that DNN learn representation in a similar way to TTN at least in this simple TTN student teacher setup with known ground truth. Also we can measure acquisition time during training as the first time at which some particular features have been learned i.e. when the  $c_{e,k}$  and acquisition score  $A_{l,e,k}^z$  are above a certain threshold (unclear what threshold they should be).

Another interesting experiment would be to establish whether a represented feature is actually useful for some computation. One approach would be to ablate one represented feature from the DNN and measure the effect on the loss. If the loss does not change the feature is represented but not causally useful in some computation.

For practical estimation of whether mode have been learned collect set of inputs, estimate features and function then compute empirical estimator of  $C$  (which replace expected value with sample mean) compute the frobenius norm between  $C$  and the matrix of singular values  $S$  for each edge  $e$ . Compute acquisition times.

For practical estimation of acquisition score we need to collect activations over some a set of inputs split them into training, validation and test probes. Fit the linear probe to the features, estimate acquisition scores. For degenerate subspace maybe CCA or multivariate regression would be useful.

Also need to define similar observable for the student TTN but should be easier because it has well defined HSVD that we could compare with the teacher.

## 5 Experiments (slop)

This section is to be completed. For reference, the experiments suggested in the previous sections are listed below. Detailed protocols, with teachers, observables, statistical analysis and decision criteria, are given in [Surrogate note \(2026, Sections 6–10\)](#) and [Local TN note \(2026, Sections 9–11\)](#).

- E0. *Algebraic check*. Verify Theorem 2.30 numerically on a tiny network in double precision: the recurrence, the tied-tree contraction and the explicit coefficient tensor must agree in value and in gradient with respect to  $\theta$ . An exactly polynomial activation, such as  $\sigma(z) = z + z^2$ , must give the same polynomial for every centre, and untying the cores must break the agreement of gradients. This check must pass before any scientific comparison.
- E1. *Global comparison* (Hypothesis 1.2). On a teacher with known features, for example the staircase teacher  $f^*(x) = \sum_m \sqrt{\lambda_m} \prod_{i \leq m} u_i^\top x^{(i)}$  on independent Gaussian blocks  $x^{(i)}$  with unit vectors  $u_i$ , train a DNN and TTNs (on the physical tree, and the tied TTN of the polynomial proxy) on the same data with the same optimizer. Compare losses, function-space distances, acquisition times and orders (1.1), and tangent kernels (Remark 1.4).
- E2. *Local surrogates* (Hypothesis 1.3). At several checkpoints, build the order- $k$  surrogates of Definition 2.43 for  $k \in \{1, 2, 3, 4\}$ ; measure  $\varepsilon_{0,j}$ ,  $\varepsilon_{1,j}$  and  $\beta_j$  at the checkpoint; run blind rollouts from the same parameters and optimizer state. Report the largest horizon over which predictions, feature coefficients and gradients agree within tolerance, as a function of  $k$ , depth and activation spread.
- E3. *Polynomial proxies* (Definition 2.51). Train proxies with  $K \in \{2, 3, 4\}$  alongside the original DNN, track  $\sigma_{\ell,j}(t)$  and  $r_\ell(t)$  to test for saddle-to-saddle (rank-incremental) dynamics, compare them with the same diagnostics for the local surrogates of the DNN, and compare feature acquisition with the DNN.
- E4. *Interpretability* (Question 2.41). Train an MLP on modular addition, with concatenated one-hot inputs and quadratic or smooth activations, for example the quadratic network of [Gromov \(2023\)](#). Compute the physical HSVD of its logit tensor (Example 2.42) and the HSVD of its computational TTN, or of its TTN surrogate for smooth activations, and compare both with the Fourier subspaces of the key frequencies.

## References

- Y. Bai, B. Krause, H. Wang, C. Xiong, and R. Socher. Taylorized training: towards better approximation of neural network training at finite width. [arXiv:2002.04010](#), 2020.
- N. Cohen, O. Sharir, and A. Shashua. On the expressive power of deep learning: a tensor analysis. In *Conference on Learning Theory (COLT)*, PMLR 49:698–728, 2016.
- C. Davis and W. M. Kahan. The rotation of eigenvectors by a perturbation. III. *SIAM Journal on Numerical Analysis*, 7(1):1–46, 1970.
- L. De Lathauwer, B. De Moor, and J. Vandewalle. A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1253–1278, 2000.

- V. de Silva and L.-H. Lim. Tensor rank and the ill-posedness of the best low-rank approximation problem. *SIAM Journal on Matrix Analysis and Applications*, 30(3):1084–1127, 2008.
- Hierarchical Feature Learning Project. Diagonal tree tensor networks: hierarchical feature learning through shared factors and residual paths. Internal research note, 30 September 2026.
- T. Dooms, W. Gauderis, G. A. Wiggins, and J. Oramas. Compositionality unlocks deep interpretable models. arXiv:2504.02667, 2025.
- S. S. Du and J. D. Lee. On the power of over-parametrization in neural networks with quadratic activation. In *International Conference on Machine Learning (ICML)*, 2018.
- C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.
- Z. Furman, S. Wäldchen, Y. Bei, and L. Hodgkinson. Benign loss landscapes can coexist with worst-case hardness. arXiv:2609.13057, 2026.
- L. Grasedyck. Hierarchical singular value decomposition of tensors. *SIAM Journal on Matrix Analysis and Applications*, 31(4):2029–2054, 2010.
- L. Grasedyck and W. Hackbusch. An introduction to hierarchical (H-) rank and TT-rank of tensors with examples. *Computational Methods in Applied Mathematics*, 11(3):291–304, 2011.
- A. Gromov. Grokking modular arithmetic. arXiv:2301.02679, 2023.
- W. Hackbusch. *Tensor Spaces and Numerical Tensor Calculus*. Springer, 2012.
- W. Hackbusch and S. Kühn. A new scheme for the tensor representation. *Journal of Fourier Analysis and Applications*, 15(5):706–722, 2009.
- J. Håstad. Tensor rank is NP-complete. *Journal of Algorithms*, 11(4):644–654, 1990.
- C. J. Hillar and L.-H. Lim. Most tensor problems are NP-hard. *Journal of the ACM*, 60(6):45, 2013.
- F. L. Hitchcock. The expression of a tensor or a polyadic as a sum of products. *Journal of Mathematics and Physics*, 6:164–189, 1927.
- S. Holtz, T. Rohwedder, and R. Schneider. On manifolds of tensors of fixed TT-rank. *Numerische Mathematik*, 120(4):701–731, 2012.
- J. Huang and H.-T. Yau. Dynamics of deep neural networks and neural tangent hierarchy. In *International Conference on Machine Learning (ICML)*, PMLR 119:4542–4551, 2020.
- A. Jacot, F. Gabriel, and C. Hongler. Neural tangent kernel: convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- A. Jacot, F. Ged, B. Şimşek, C. Hongler, and F. Gabriel. Saddle-to-saddle dynamics in deep linear networks: small initialization training, symmetry, and sparsity. arXiv:2106.15933, 2021.
- T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- J. M. Landsberg, Y. Qi, and K. Ye. On the geometry of tensor network states. *Quantum Information and Computation*, 12(3–4):346–354, 2012.
- L.-H. Lim. Tensors in computations. *Acta Numerica*, 30:555–764, 2021.
- Hierarchical Feature Learning Project. Deep hierarchical feature learning: literature review and research agenda. Internal project document, 2026.

- Hierarchical Feature Learning Project. Local polynomial tensor networks for deep neural network training: exact tree representations, controlled changes of local model, and a teacher–student experimental protocol. Internal research note, 1 October 2026.
- P. Morala, J. A. Cifuentes, R. E. Lillo, and I. Ucar. NN2Poly: a polynomial representation for deep feed-forward artificial neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 36(1):781–795, 2025.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2301.05217.
- I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.
- M. T. Pearce, T. Dooms, A. Rigg, J. M. Oramas, and L. Sharkey. Bilinear MLPs enable weight-based mechanistic interpretability. arXiv:2410.08417, 2024.
- S. Pesme and N. Flammarion. Saddle-to-saddle dynamics in diagonal linear networks. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177, 2022.
- N. Razin, A. Maman, and N. Cohen. Implicit regularization in hierarchical tensor factorization and deep convolutional neural networks. In *International Conference on Machine Learning (ICML)*, PMLR 162:18422–18462, 2022.
- O. Roy and M. Vetterli. The effective rank: a measure of effective dimensionality. In *European Signal Processing Conference (EUSIPCO)*, pages 606–610, 2007.
- S. Sarao Mannelli, E. Vanden-Eijnden, and L. Zdeborová. Optimization and generalization of shallow neural networks with quadratic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations (ICLR)*, 2014. arXiv:1312.6120.
- A. M. Saxe, J. L. McClelland, and S. Ganguli. A mathematical theory of semantic development in deep neural networks. *Proceedings of the National Academy of Sciences*, 116(23):11537–11546, 2019.
- Y.-Y. Shi, L.-M. Duan, and G. Vidal. Classical simulation of quantum many-body systems with a tree tensor network. *Physical Review A*, 74:022320, 2006.
- E. M. Stoudenmire and D. J. Schwab. Supervised learning with tensor networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Hierarchical Feature Learning Project. Tree tensor networks as global and local surrogates for deep neural network training dynamics: mathematical construction and a staged empirical program. Internal research note, 2 October 2026.
- L. R. Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3):279–311, 1966.
- A. Uschmajew and B. Vandereycken. The geometry of algorithms using hierarchical tensors. *Linear Algebra and its Applications*, 439(1):133–166, 2013.
- P.-Å. Wedin. Perturbation bounds in connection with singular value decomposition. *BIT Numerical Mathematics*, 12(1):99–111, 1972.
- Y. Zhang, A. M. Saxe, and P. E. Latham. Saddle-to-saddle dynamics explains a simplicity bias across neural network architectures. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2512.20607.